

Machine learning-assisted multi-time-step method for structural dynamics

Sun-Beom Kwon¹ and Arun Prakash²

¹Assistant Professor, Department of Naval Architecture, Ocean and Mobility Engineering, Hongik University, Sejong 30016, Republic of Korea (corresponding author). Email:

sbkwon@hongik.ac.kr

²Associate Professor, Lyles School of Civil and Construction Engineering, Purdue University, West Lafayette, IN 47907. Email: aprakas@purdue.edu

ABSTRACT

Multi-time-step (MTS) methods enable fast, stable, and accurate simulations of multi-scale problems in structural dynamics. This is achieved by decomposing a large computer model into several subdomains and using a small time step in subdomains with high temporal gradients while integrating other subdomains with a large time step to save computational cost. Thus, compared to conventional uniform time-stepping (UTS) schemes that use a small time step for the entire problem domain, MTS methods are significantly faster. Despite the computational efficiency of existing MTS methods, they still require a 3-step process: solve the subdomains independently, compute the Lagrange multipliers needed for enforcing continuity of the solution across the interface between subdomains, and finally couple all the subdomains together by updating their solutions to satisfy the continuity constraints. The key to obtaining accurate solutions with MTS methods is to compute the Lagrange multipliers accurately, which necessitates the time-consuming 3-step process. To eliminate this 3-step process we use a machine learning approach to obtain the Lagrange multipliers needed for coupling the various subdomains at each time step. Data collected from solutions generated using conventional MTS methods are used to train a long short-term memory (LSTM) network that predicts these Lagrange multipliers. Output from the LSTM network, along

with other state information, is then used to solve all the subdomains in a single (1-step) process. The resulting algorithm is called ML-MTS and it advances the global solution in a single pass while maintaining continuity of the solution across the subdomain interfaces. Two numerical examples are presented to compare the performance of the proposed ML-MTS approach to existing MTS methods in terms of accuracy and computational cost. Results show that the proposed ML-MTS method achieves additional speedup on top of the computational gains of existing MTS methods while maintaining the accuracy of the solution.

Author keywords: Structural dynamics, Domain decomposition, Multi-time-step, Long short-term memory (LSTM) network, Time integration

INTRODUCTION

In finite element analysis, direct time integration is used to solve dynamic equilibrium equations for structural dynamics and wave propagation in solids (Chopra 2012; Zienkiewicz and Taylor 2005; Hughes 2000; Bathe 2016). Direct time integration can be categorized into implicit and explicit schemes. Implicit schemes use a larger time step and several are unconditionally stable. On the other hand, explicit schemes are usually used for impact, crash, and short-time wave propagation problems because they have low computational cost per time step. However, they are associated with a maximum critical time step given by the Courant-Friedrichs-Lewy (CFL) condition (Courant et al. 1928).

Conventionally, time integration schemes use a uniform time step (UTS) for an entire problem domain. However, for problems that have multiple temporal scales – when it is necessary to capture both local transients and global dynamics of a structure – UTS schemes are not well-suited. For such problems, UTS schemes must meet the most stringent stability and accuracy requirements and thereby they are constrained to use a small time step for the entire problem domain, resulting in a large computational cost. To achieve high overall accuracy at a low computational cost, researchers have proposed a variety of sub-cycling methods (Daniel 1998; Miao et al. 2008), space-time methods (Hulbert and Hughes 1990; Wiberg and Li 1999; Abedi et al. 2010; Subber and Matouš 2016), asynchronous methods (Lew et al. 2003; Gates et al. 2008; Seibold and Rixen 2019),

and other multi-rate and multi-scale methods (Bottasso 2002; Waisman and Fish 2008; Crouch and Oskay 2015) that circumvent some of the limitations of UTS methods. However, computer implementation of these methods is usually quite intrusive making it difficult to leverage existing legacy code that has been optimized for conventional UTS methods. An attractive alternative to these methods are multi-time-step (MTS) methods in conjunction with domain partitioning that allow the spatial problem domain to be partitioned into subdomains and enable the use of existing time integration schemes to advance each subdomain with a time step that is suitable to capture the local dynamics of the problem.

Partitioning methods can be categorized into two groups: 1) node partitioning methods (Belytschko and Mullen 1976; Belytschko and Mullen 1978; Belytschko et al. 1979) and 2) element partitioning methods (Hughes and Liu 1978a; Hughes and Liu 1978b). For node partitioning methods, different nodes are assigned to different subdomains. They generally lead to a complex coupling algorithm with high computational costs (Smolinski et al. 1988). On the other hand, element partitioning methods assign different elements to different subdomains, which leads to symmetric system matrices (e.g. symmetric mass and stiffness matrices). In addition, with element partitioning, there are shared nodes on the interface between subdomains, which enables each subdomain to be solved independently of each other. Thus, element partitioning methods can be used for different element types (e.g. beam, shell, and solid) or different media (e.g. structure, fluid, and soil) (Park 1980).

When MTS methods are used together with partitioning, a problem domain is divided into smaller subdomains, and different time integration schemes and/or time steps may be used to meet the stability and accuracy requirements of individual subdomains. Several MTS methods have been proposed that either use a single time step in different subdomains (Belytschko and Mullen 1976; Hughes and Liu 1978b; Hughes and Liu 1978a; Park 1980) or use different time steps in different subdomains (Belytschko et al. 1979; Liu and Belytschko 1982; Belytschko et al. 1984; Smolinski 1992; Daniel 1997; Bursi et al. 2010). Farhat and coworkers (Farhat and Roux 1991; Farhat and Roux 1994; Farhat et al. 1994) presented a dual-Schur domain decomposition (DD)

method that uses element partitioning and Lagrange multipliers to enforce kinematic continuity at the interface of shared nodes between subdomains. Based on the dual-Schur DD method, MTS methods have been proposed to couple different time integration schemes with different time steps (Gravouil and Combescure 2001; Combescure and Gravouil 2002; Prakash and Hjelmstad 2004; Prakash et al. 2014; Mahjoubi et al. 2011; Brun et al. 2015; Gravouil et al. 2015; Mahjoubi and Krenk 2010; Karimi and Nakshatrala 2014; Kwon and Prakash 2025a; Kwon and Prakash 2025b). However, existing MTS methods require a computationally intensive 3-step process at each time step to advance the solution: 1) solve each subdomain independently; 2) compute Lagrange multipliers on the interface by imposing continuity constraints; and 3) update the solutions using the computed Lagrange multipliers. Despite the computational advantages of existing MTS methods over conventional UTS methods, this 3-step process limits the achievable speedup in the simulation.

In recent years, machine learning (ML) techniques, such as artificial neural networks (ANN), have been applied to various fields in structural mechanics, including structural health monitoring (Farrar and Worden 2012) and structural system identification (Chiuso and Pillonetto 2019), and structural dynamics (Cunha et al. 2023). For instance, feedforward neural networks (FFNs), also known as multilayer perceptrons (MLPs), have been often used to predict dynamic responses of structures (Guarize et al. 2007; Lagaros and Papadrakakis 2012). Since FFNs are the simplest type of neural networks, they can fail to capture complex nonlinearities in structural dynamics. As alternatives, convolutional neural networks (CNNs), which are well-suited for handling data with a grid-like structure, can be used for estimating structural dynamic response (LeCun et al. 1995; Zhang et al. 2020a). Additionally, recurrent neural networks (RNNs), designed to model sequential data, are widely used to capture the dynamic response of structures (Williams and Zipser 1989). Among RNN variations, long short-term memory (LSTM) networks have demonstrated superior performance in predicting long sequences in structural dynamics (Zhang et al. 2019; Zhang et al. 2020b; Xiang et al. 2024).

In this paper, we present a new type of MTS approach that uses ML to eliminate the cumbersome 3-step process of conventional MTS methods. In the proposed ML-MTS method, a deep LSTM

network predicts the time history of Lagrange multipliers on the interface between subdomains *a priori*. The predicted Lagrange multipliers are then used in conjunction with any external forces to advance each subdomain in a single (1-step) process, while ensuring that the computed solution maintains continuity across the subdomain interfaces. To develop the proposed ML-MTS method, we first review existing MTS methods that are based on a dual-Schur DD method. We present the architecture of a deep LSTM network, along with its training and choice of hyperparameters, that is used for predicting Lagrange multipliers. Next, we describe the algorithmic formulation of how the predicted Lagrange multipliers are used to advance the subdomain solutions in a single-step. Finally, the performance of the proposed ML-MTS method is compared to that of existing MTS methods using two representative numerical examples.

MULTI-TIME-STEP (MTS) METHODS

In this section, we review existing multi-time-step (MTS) methods that are based on dual-Schur domain decomposition (DD) – see (Kwon and Prakash 2025b) for details. The semi-discrete governing differential equations of motion for linear elasto-dynamics of a problem domain Ω are written as:

$$\mathbf{M}\ddot{\mathbf{u}}(t) + \mathbf{D}\dot{\mathbf{u}}(t) + \mathbf{K}\mathbf{u}(t) = \mathbf{f}(t) \text{ in } \Omega \quad (1)$$

where $\mathbf{M}$, $\mathbf{D}$, $\mathbf{K}$, $\mathbf{u}$, and $\mathbf{f}$ denote the mass, damping, stiffness, nodal displacements, and external nodal forces, respectively and superimposed dot ($\dot{\cdot}$) denotes time derivative. The problem domain is decomposed into S subdomains $\Omega^{(1)}, \Omega^{(2)}, \dots, \Omega^{(S)}$ with an interface Γ^I between them, as shown in Fig. 1. Using a dual-Schur DD, the partitioned governing equations for subdomain $\Omega^{(s)}$ are given as:

$$\mathbf{M}^{(s)}\ddot{\mathbf{u}}^{(s)}(t) + \mathbf{D}^{(s)}\dot{\mathbf{u}}^{(s)}(t) + \mathbf{K}^{(s)}\mathbf{u}^{(s)}(t) + \mathbf{C}^{(s)T}\boldsymbol{\lambda}(t) = \mathbf{f}^{(s)}(t) \text{ in } \Omega^{(s)} \quad (2)$$

where $\mathbf{C}^{(s)}$ represents the connectivity matrix between subdomain $\Omega^{(s)}$ and the interface Γ^I , and $\boldsymbol{\lambda}$ denotes the vector of Lagrange multipliers needed to impose continuity between subdomains.

Upon time integration, the semi-discrete Eq. (2) is transformed into a system of algebraic equations. For ease of explanation, we only show the algebraic equations using the Newmark

scheme (Newmark 1959) with two parameters, $\gamma^{(s)}$ and $\beta^{(s)}$, and using a time step $\Delta t^{(s)}$ to advance the solution in $m^{(s)}$ steps from a known state at t_n to $t_{n+1} = t_n + \Delta T$ as:

$$\mathbb{M}^{(s)} \mathbb{X}_{n+j/m}^{(s)} + \mathbb{C}^{(s)T} \lambda_{n+j/m}^{(s)} = \mathbb{f}^{(s)} - \mathbb{N}^{(s)} \mathbb{X}_{n+(j-1)/m}^{(s)} \text{ for } j = 1, 2, \dots, m^{(s)} \text{ and } s = 1, 2, \dots, S \quad (3)$$

where ΔT is the global block time step for all subdomains and $m^{(s)}$ is the time-step ratio for subdomain $\Omega^{(s)}$ with $m^{(s)} = \Delta T / \Delta t^{(s)}$. The other block matrices in Eq. (3) are defined as:

$\mathbb{C} = \begin{bmatrix} \mathbf{C} & \mathbf{0} & \mathbf{0} \end{bmatrix}$ and

$$\mathbb{M} = \begin{bmatrix} \mathbf{M} & \mathbf{D} & \mathbf{K} \\ -\gamma \Delta t \mathbf{I} & \mathbf{I} & \mathbf{0} \\ -\beta \Delta t^2 \mathbf{I} & \mathbf{0} & \mathbf{I} \end{bmatrix}, \quad \mathbb{N} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} \\ (\gamma - 1) \Delta t \mathbf{I} & -\mathbf{I} & \mathbf{0} \\ (\beta - 0.5) \Delta t^2 \mathbf{I} & -\Delta t \mathbf{I} & -\mathbf{I} \end{bmatrix}, \quad \mathbb{X} = \begin{bmatrix} \mathbf{a} \\ \mathbf{v} \\ \mathbf{d} \end{bmatrix}, \quad \mathbb{f} = \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix} \quad (4)$$

where $\mathbf{a}_n$, $\mathbf{v}_n$, and $\mathbf{d}_n$ are the discretized acceleration, velocity, and displacement, respectively, at t_n . Note that matrices $\mathbb{M}$, $\mathbb{N}$, and vector $\mathbb{f}$ in Eq. (4) depend upon the specific time integration scheme used. A more general formulation that works for any time integration scheme is provided in (Kwon and Prakash 2025b).

Next, continuity of the solution across the subdomain interfaces is ensured using Lagrange multipliers that enforce the constraint conditions:

$$\sum_{s=1}^S \mathbb{B}^{(s)} \mathbb{X}^{(s)} = \mathbf{0} \quad (5)$$

where $\mathbb{B}$ is defined as $\begin{bmatrix} \mathbf{0} & \mathbf{C} & \mathbf{0} \end{bmatrix}$ for velocity continuity (VC) (Prakash and Hjelmstad 2004), $\begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{C} \end{bmatrix}$ for displacement continuity (DC) (Mahjoubi and Krenk 2010), and $\begin{bmatrix} \mathbf{0} & \mathbf{C} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{C} \end{bmatrix}$ for velocity and displacement continuity (VCDC) (Kwon and Prakash 2025b).

To ensure numerical stability, most MTS methods in the literature impose velocity continuity (VC) across subdomain interfaces at block time step ΔT intervals and use linear interpolation for

Lagrange multipliers between t_n and t_{n+1} :

$$\lambda_{n+j/m}^{(s)} = \left(1 - \frac{j}{m^{(s)}}\right) \lambda_n + \frac{j}{m^{(s)}} \lambda_{n+1} \quad (6)$$

where λ_n and λ_{n+1} are Lagrange multipliers at t_n and t_{n+1} , respectively.

Eqs. (3), (5), and (6) are solved using the Schur complement method. First, we compute the uncoupled-free part in subdomain $\Omega^{(s)}$:

$$\bar{\mathbb{x}}_{n+j/m}^{(s)} = (\mathbb{M}^{(s)})^{-1} \left(\mathbb{F}_{n+j/m}^{(s)} - \mathbb{N}^{(s)} \bar{\mathbb{x}}_{n+(j-1)/m}^{(s)} - \left(1 - \frac{j}{m^{(s)}}\right) \mathbb{C}^{(s)T} \lambda_n \right) \text{ for } j = 1, 2, \dots, m^{(s)} \quad (7)$$

with $\bar{\mathbb{x}}_n = \mathbb{x}_n$. Then, Lagrange multipliers on the interface are computed as

$$\lambda_{n+1} = \left(\sum_{s=1}^N \sum_{j=1}^{m^{(s)}} \mathbb{B}^{(s)} \mathbb{Y}_{j/m}^{(s)} \right)^{-1} \left(\sum_{s=1}^N \sum_{j=1}^{m^{(s)}} \mathbb{B}^{(s)} \bar{\mathbb{x}}_{n+j/m}^{(s)} \right) \quad (8)$$

where

$$\mathbb{Y}_{j/m}^{(s)} = (\mathbb{M}^{(s)})^{-1} \left(\frac{j}{m^{(s)}} \mathbb{C}^{(s)T} - \mathbb{N}^{(s)} \mathbb{Y}_{(j-1)/m}^{(s)} \right) \quad (9)$$

with $\mathbb{Y}_0^{(s)} = \mathbf{0}$. Finally, we update solutions to incorporate the coupled-link correction as

$$\mathbb{x}_{n+j/m}^{(s)} = \bar{\mathbb{x}}_{n+j/m}^{(s)} - \mathbb{Y}_{j/m}^{(s)} \lambda_{n+1} \text{ for } j = 1, 2, \dots, m^{(s)} \text{ and } s = 1, 2, \dots, S \quad (10)$$

Thus, existing MTS methods compute the solution using Eqs. (7), (8), and (10) sequentially, thereby requiring a time-consuming 3-step process.

NOVEL MACHINE LEARNING-ASSISTED (ML-MTS) METHOD

In this section, we employ a deep LSTM network to predict Lagrange multipliers directly without solving Eqs. (7) and (8). Since MTS solutions are sensitive to Lagrange multipliers, we propose a new strategy to couple subdomains by imposing continuity of all kinematic quantities, instead of directly updating the subdomain solutions using the predicted Lagrange multipliers.

Deep LSTM network

Fig. 2a shows the architecture of a deep LSTM network composed of multiple hidden LSTM layers and one dense layer. Between LSTM layers, there are additional activation functions. The network maps an input time series of external loadings (such as ground acceleration and external forces) to an output time series of Lagrange multipliers on the interface. The LSTM layer consists of multiple LSTM cells, which are analogous to artificial neurons in FFNs. As shown in Fig. 2b, the forward pass of an LSTM cell can be expressed as

$$\mathbf{g}_n = \sigma \left(\mathbf{W}^g \begin{bmatrix} \ddot{\mathbf{x}}_{g,n} \\ \boldsymbol{\ell}_{n-1} \end{bmatrix} + \mathbf{b}^g \right), \quad \mathbf{i}_n = \sigma \left(\mathbf{W}^i \begin{bmatrix} \ddot{\mathbf{x}}_{g,n} \\ \boldsymbol{\ell}_{n-1} \end{bmatrix} + \mathbf{b}^i \right) \quad (11)$$

$$\mathbf{o}_n = \sigma \left(\mathbf{W}^o \begin{bmatrix} \ddot{\mathbf{x}}_{g,n} \\ \boldsymbol{\ell}_{n-1} \end{bmatrix} + \mathbf{b}^o \right), \quad \tilde{\mathbf{h}}_n = \tanh \left(\mathbf{W}^{\tilde{h}} \begin{bmatrix} \ddot{\mathbf{x}}_{g,n} \\ \boldsymbol{\ell}_{n-1} \end{bmatrix} + \mathbf{b}^{\tilde{h}} \right) \quad (12)$$

$$\mathbf{h}_n = \mathbf{g}_n * \mathbf{h}_{n-1} + \mathbf{i}_n * \tilde{\mathbf{h}}_n, \quad \boldsymbol{\ell}_n = \mathbf{o}_n * \tanh(\mathbf{h}_n) \quad (13)$$

where $\ddot{\mathbf{x}}_{g,n}$ is input and $\boldsymbol{\ell}_n$ is output at time t_n . Vectors $\mathbf{g}_n$, $\mathbf{i}_n$, $\mathbf{o}_n$, and $\mathbf{h}_n$ denote the forget gate, input gate, output gate, and hidden state, respectively, at time t_n . Matrices $\mathbf{W}^g$, $\mathbf{W}^i$, $\mathbf{W}^o$, and $\mathbf{W}^{\tilde{h}}$ are the corresponding weight matrices and vectors $\mathbf{b}^g$, $\mathbf{b}^i$, $\mathbf{b}^o$, and $\mathbf{b}^{\tilde{h}}$ are the corresponding bias vectors. The operator $*$ denotes the element-wise product, and σ and $\tanh$ are the sigmoid and hyperbolic tangent functions, respectively.

Formulation of ML-MTS method

Utilizing the same decomposition of the problem domain Ω into S subdomains as described in the previous section, we assume, without loss of generality, that subdomain $\Omega^{(S)}$ is the stiffest among all subdomains i.e. $\Delta t^{(S)} \leq \Delta t^{(s)}$ for any value of s . If there are multiple subdomains with the smallest time step, then one may pick any of them randomly. Note that smaller time steps are used to capture local dynamic behavior in stiffer subdomains because in a stiff subdomain, the maximum natural frequency is higher than that of a flexible subdomain. This may result from physical factors (e.g. material properties, geometry, boundary conditions etc.) or numerical factors

(e.g. rigid connections, constraints, penalty factors etc.) - see (Bathe and Noh 2012; Kwon and Prakash 2025a) for details.

In the proposed ML-MTS method, Lagrange multipliers at $t_{n+j/m}^{(S)}$ on interface Γ^I are predicted using a deep LSTM network when external loadings are fed as input to the network. For instance,

$$\ddot{x}_g(t_{n+j/m}^{(S)}) \xrightarrow{\text{Deep LSTM network}} \lambda(t_{n+j/m}^{(S)}) \text{ for } j = 1, 2, \dots, m^{(S)} \text{ and } n = 1, 2, \dots, N^{(S)} \quad (14)$$

where $\ddot{x}_g$ is the ground acceleration and $N^{(S)}$ is the number of $\Delta t^{(S)}$ time steps in the entire simulation. Note that $N^{(S)} = T/\Delta t^{(S)}$, where T is the duration of the simulation. Using an interpolation scheme of choice (e.g. Lagrange, spline, etc.), one can obtain Lagrange multipliers at any time t :

$$\lambda(t) = \sum_{n=1}^{N^{(S)}} \sum_{j=1}^{m^{(S)}} w_{n+j/m}^{\lambda}(t) \lambda(t_{n+j/m}^{(S)}) \quad (15)$$

where $w_{n+j/m}^{\lambda}(t)$ is a user-defined interpolation function (Kwon and Prakash 2025b).

Note that, once the predicted Lagrange multipliers are obtained from the LSTM network, one can directly update the partitioned set of ordinary differential equations (Eq. (3)) - we call this approach the vanilla ML-MTS method. However, the vanilla ML-MTS method leads to large drifts and errors in the kinematic quantities (displacements, velocities and accelerations) – as shown later in Fig. 5. To overcome this challenge, we propose a novel ML-MTS algorithm that imposes continuity of *all* kinematic quantities simultaneously as explained next.

The ML-MTS method begins by solving the stiffest subdomain $\Omega^{(S)}$ first. To determine the sequence of solution for the remaining subdomains $\Omega^{(s)}$ ($1 \leq s \leq S-1$), we create two sets of subdomains denoted as Ω_{solved} and $\Omega_{neighbor}$, as shown in Fig. 3. Note that the set Ω_{solved} initially contains only $\Omega^{(S)}$ (as it has been solved now) and the set $\Omega_{neighbor}$ contains all the subdomains that share an interface with any of the subdomains in Ω_{solved} . Next, we identify subdomain $\Omega^{(s)}$ with the smallest time step in $\Omega_{neighbor}$ and its interface $\Gamma^{(s,q)}$ with all subdomains $\Omega^{(q)}$ that are in Ω_{solved} and are adjacent to $\Omega^{(s)}$. Now, subdomain $\Omega^{(s)}$ is solved in a single (1-step) process using

Eq. (3) as:

$$\mathbb{X}_{n+j/m}^{(s)} = \left(\mathbb{M}^{(s)} \right)^{-1} \left(\mathbb{F}_{n+j/m}^{(s)} - \mathbb{C}^{(s)T} \lambda(t_{n+j/m}^{(s)}) - \mathbb{N}^{(s)} \mathbb{X}_{n+(j-1)/m}^{(s)} \right) \text{ in } \Omega^{(s)} \text{ for } j = 1, 2, \dots, m^{(s)} \quad (16)$$

while enforcing the boundary condition $\mathbb{X}_{n+j/m}^{(s)} = \mathbb{X}^{(q)}(t_{n+j/m}^{(s)})$ on $\Gamma^{(s,q)}$ and using the predicted Lagrange multipliers $\lambda(t_{n+j/m}^{(s)})$ from the LSTM network as external forces on the remaining interfaces. Note that $\mathbb{X}^{(q)}$ can be obtained at $t_{n+j/m}^{(s)}$ using any interpolation scheme:

$$\mathbb{X}^{(q)}(t_{n+j/m}^{(s)}) = \sum_{k=0}^{m^{(q)}} \mathbb{W}_k(t_{n+j/m}^{(s)}) \mathbb{X}_{n+k/m}^{(q)} \quad (17)$$

where $\mathbb{W}_k(t)$ consists of user-defined interpolation functions for kinematic quantities (Kwon and Prakash 2025b). Finally, Ω_{solved} is updated by adding $\Omega^{(s)}$ to it and checking if the entire problem domain Ω has been solved. If not, then the entire process is repeated to solve the next subdomain, whereas if $\Omega_{solved} = \Omega$, then the simulation progresses to the next block time step ΔT . Algorithm 1 summarizes the entire procedure for the proposed ML-MTS method.

Note that when $\Omega^{(q)} = \emptyset$ is chosen for all subdomains (i.e. no continuity conditions between subdomains), the proposed ML-MTS method is identical to the vanilla ML-MTS method as a special case. Further, note that the stability of the proposed ML-MTS method depends on the stability of the time integration schemes used for each subdomain and on the predicted Lagrange multipliers from the LSTM network. Lastly, note that even though the solution process for the ML-MTS method requires a sequential solution of subdomains, the method still affords the ability to parallelize this process by *pipelining* (Lee et al. 2015). For instance, once subdomain $\Omega^{(s)}$ is advanced from t_n to t_{n+1} , then it need not wait for all the remaining subdomains to be advanced to t_{n+1} as well. It can proceed to the next time step t_{n+2} as soon as subdomains $\Omega^{(q)}$, with whom it shares the interface $\Gamma^{(s,q)}$ on which boundary conditions are prescribed, are advanced to t_{n+2} . In this manner, all the subdomains in Ω may be solved in parallel, albeit in a staggered pipeline to ensure that the sequence of subdomain solution and any dependencies between them are preserved.

Algorithm 1 Solution procedure for the proposed ML-MTS method

A. Initialization

1. Decompose the spatial domain into $\Omega^{(1)}, \Omega^{(2)}, \dots, \Omega^{(S)}$, and select time-step sizes $\Delta t^{(s)}$ such that $\Delta t^{(S)} \leq \Delta t^{(s)} \forall s$ and the corresponding time integration scheme for each subdomain $\Omega^{(s)}$.
2. Construct mass matrices $\mathbf{M}^{(s)}$, damping matrices $\mathbf{D}^{(s)}$, stiffness matrices $\mathbf{K}^{(s)}$, connectivity matrices $\mathbf{C}^{(s)}$, and force vectors $\mathbf{f}^{(s)}$.
3. Initialize $\mathbf{d}_0^{(s)}$, $\mathbf{v}_0^{(s)}$, and $\mathbf{a}_0^{(s)}$.
4. Predict the Lagrange multipliers over the simulation interval $[0, T]$ using the Deep-LSTM network: e.g. $\ddot{\mathbf{x}}_g(t_{n+j/m}^{(S)}) \xrightarrow{\text{Deep LSTM network}} \boldsymbol{\lambda}(t_{n+j/m}^{(S)})$

B. Ordering of subdomain solutions

1. Solve the stiffest subdomain $\{\Omega^{(S)}\}$ first
2. Initialize $\Omega_{solved} = \Omega^{(S)}$ and $\Omega_{neighbor} = \{\Omega^{(k)} \notin \Omega^{(S)} | \Omega^{(k)} \text{ shares an interface with } \Omega^{(S)}\}$
3. **while** $|\Omega_{solved}| < S$
 Select $\Omega^{(s)} = \arg \min_{\Omega^{(k)} \in \Omega_{neighbor}} \Delta t^{(k)}$
 Identify interfaces $\Gamma^{(s,q)}$ with all adjacent subdomains $\Omega^{(q)} \in \Omega_{solved}$
 Add $\Omega^{(s)}$ to Ω_{solved}
 Update $\Omega_{neighbor} = \{\Omega^{(k)} \notin \Omega_{solved} | \Omega^{(k)} \text{ shares an interface with any } \Omega^{(q)} \in \Omega_{solved}\}$
end while

C. For each cycle of the block time step ΔT **for** each subdomain $\Omega^{(s)}$ in the prescribed order**for** $j = 1, 2, \dots, m^{(s)}$ Enforce $\mathbf{x}_{n+j/m}^{(s)} = \mathbf{x}^{(q)}(t_{n+j/m}^{(s)})$ on $\Gamma^{(s,q)}$ Apply the predicted Lagrange multipliers $\boldsymbol{\lambda}(t_{n+j/m}^{(s)})$ as external interface forces on the remaining interfacesCompute $\mathbf{x}_{n+j/m}^{(s)} = \left(\mathbb{M}^{(s)}\right)^{-1} \left(\mathbf{f}_{n+j/m}^{(s)} - \mathbf{C}^{(s)T} \boldsymbol{\lambda}(t_{n+j/m}^{(s)}) - \mathbb{N}^{(s)} \mathbf{x}_{n+(j-1)/m}^{(s)}\right)$ **end for****end for**

NUMERICAL EXAMPLES

We present two representative numerical examples to demonstrate the performance of the proposed ML-MTS approach. Both numerical examples were performed in MATLAB on a Linux desktop with an NVIDIA GeForce RTX 4080 SUPER (PCIe/SSE2, CUDA 8.9), a 32-core CPU, and 126 GB RAM. For training, we generate synthetic data from white Gaussian noise samples. This ensures robustness of the method to different unknown forcing excitations and also eliminates the potential for bias to known earthquake records. The random number generator seed is fixed

using $\text{rng}(1)$ prior to sample white noise in generating data. The same seed is also used for the data splitting and subsequent training of the LSTM network.

The architecture of the LSTM network is determined by first training a model consisting of one LSTM layer and one dense layer and comparing the final losses for 2, 4, 8, 16, 32, 64, and 128 cells in the LSTM layer. We found that the final loss decreases with increasing number of cells, but remain relatively uniform after 32 cells. Next, using 32 LSTM cells per layer, we compare final losses as the number of LSTM layers is increased from 1, 2, 3, 4, 5, 6, and 7 – with and without ReLU activation functions between layers. We found 5 LSTM layers with ReLU activation function leads to smallest final loss and adopted 5 LSTM layers for our ML-MTS method.

Three degree-of-freedom problem with three subdomains

In this section, we solve a three-degree-of-freedom (3-DOF) spring-mass system consisting of three discrete masses $m_1 = 1$, $m_2 = 1$, and $m_3 = 1$, interconnected by springs $k_1 = 1$ and $k_2 = 5$ as shown in Fig. 4. Mass m_3 is connected to a fixed wall through a spring of $k_3 = 10$, while mass m_1 is subjected to an external load $f_1(t)$. The system is decomposed into three subdomains (i.e. $S = 3$) with two interface forces λ_1 and λ_2 , with nodal masses distributed equally between subdomains. The governing equations in the resulting subdomains are

$$\begin{bmatrix} m_1 & 0 \\ 0 & m_2/2 \end{bmatrix} \begin{bmatrix} \ddot{u}_1^{(1)}(t) \\ \ddot{u}_2^{(1)}(t) \end{bmatrix} + \begin{bmatrix} k_1 & -k_1 \\ -k_1 & k_1 \end{bmatrix} \begin{bmatrix} u_1^{(1)}(t) \\ u_2^{(1)}(t) \end{bmatrix} + \begin{bmatrix} 0 \\ \lambda_1 \end{bmatrix} = \begin{bmatrix} f_1(t) \\ 0 \end{bmatrix} \quad \text{in } \Omega^{(1)} \quad (18)$$

$$\begin{bmatrix} m_2/2 & 0 \\ 0 & m_3/2 \end{bmatrix} \begin{bmatrix} \ddot{u}_2^{(2)}(t) \\ \ddot{u}_3^{(2)}(t) \end{bmatrix} + \begin{bmatrix} k_2 & -k_2 \\ -k_2 & k_2 \end{bmatrix} \begin{bmatrix} u_2^{(2)}(t) \\ u_3^{(2)}(t) \end{bmatrix} - \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \text{in } \Omega^{(2)} \quad (19)$$

$$\begin{bmatrix} m_3/2 \end{bmatrix} \begin{bmatrix} \ddot{u}_3^{(3)}(t) \end{bmatrix} + \begin{bmatrix} k_3 \end{bmatrix} \begin{bmatrix} u_3^{(3)}(t) \end{bmatrix} + \begin{bmatrix} \lambda_2 \end{bmatrix} = 0 \quad \text{in } \Omega^{(3)} \quad (20)$$

We impose zero initial displacement and velocity and solve for the response using the VC-MTS, vanilla ML-MTS, and proposed ML-MTS methods. The underlying time integration scheme is the implicit Newmark scheme with $\gamma^{(s)} = 0.5$ and $\beta^{(s)} = 0.25$ (i.e. trapezoidal rule).

We use a deep LSTM network consisting of five hidden LSTM layers and one dense layer to

predict the two Lagrange multipliers (λ_1 and λ_2), under the external force $f_1(t)$. Each LSTM layer is composed of 32 LSTM cells with the ReLU (rectified linear unit) activation function between layers. For training the deep LSTM network, we generate synthetic data obtained using the VC-MTS method with $\Delta t^{(1)} = \Delta t^{(2)} = \Delta t^{(3)} = 0.02$ over a time duration of $T = 10$, subjected to 1000 random white Gaussian noise samples with a sampling frequency of 50. We note that for this choice of time steps, the results of VC-MTS and VCDC-MTS methods are identical. In general, the training can be conducted with data from any stable MTS method. We normalize the synthetic data to have zero mean and unit variance. The data are split into 80 % for training and 20 % for validation. We shuffle the training and validation datasets at the beginning of each epoch and employ the mean square error as a loss function. We use a mini-batch size of 128 and train the network for 10,000 epochs using the Adam (adaptive moment estimation) optimizer (Kingma and Ba 2014), with default values of hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\hat{\epsilon} = 10^{-8}$. The initial learning rate α is set to 10^{-3} and reduced to 10^{-4} after 5,000 epochs after training loss becomes relatively constant.

To test the performance of the proposed ML-MTS method, we consider the sinusoidal excitation force $f_1(t) = \sin(4\pi t)$ over a time duration of $T = 20$. Two measures of error are defined to quantify the accuracy of the various MTS methods: local instantaneous error ϵ_n^i and average simulation error E :

$$\epsilon_n^i(\mathbf{x}) = \frac{|x_n^i - \tilde{x}_n^i|}{\max_{i,n} \tilde{x}_n^i - \min_{i,n} \tilde{x}_n^i}, \quad E(\mathbf{x}) = \frac{\sum_i \sum_n \epsilon_n^i(\mathbf{x})}{N_i \times N} \quad (21)$$

where i denotes the degree of freedom (DOF) ($i = 1, 2, \dots, N_i$), and N_i and N are the total number of DOFs and time steps, respectively. The variable x_n^i and $\tilde{x}_n^i$ denote the numerical and reference solutions, respectively, at a DOF i at time t_n . In addition, we define the maximum drift D as:

$$D(\mathbf{d}) = \left\| \sum_{s=1}^S \mathbf{C}^{(s)} \mathbf{d}^{(s)} \right\|_{\infty} \quad (22)$$

where $\|\cdot\|_{\infty}$ denotes the L-infinity norm.

Fig. 5(a) and (b) show the time history of the Lagrange multipliers at nodes 1 and 2 using the

VC-MTS and ML-MTS methods with $\Delta t^{(1)} = 2\Delta t^{(2)} = 5\Delta t^{(3)} = 1/10$. The reference solution for this problem is obtained using the VC-MTS method with a small time step of $\Delta t^{(1)} = \Delta t^{(2)} = \Delta t^{(3)} = 0.002$. We note that the deep LSTM network produces accurate predictions for the Lagrange multipliers. Fig. 5(c)–(f) present the time history of nodal displacements using the VC-MTS, the vanilla ML-MTS, and the proposed ML-MTS methods. For Fig. 5(c)–(f), the numerical solution obtained using the UTS (Newmark) method with a small time step of 5×10^{-5} is taken to be the reference solution. Fig. 5 also lists errors and drift in displacements for the VC-MTS, the vanilla ML-MTS, and the proposed ML-MTS methods. While the vanilla ML-MTS method leads to large errors as well as large drifts in displacements, the proposed ML-MTS method produces an accurate solution with levels of error that are similar to the conventional MTS method.

Nine-story building subject to earthquake

In this section, we simulate a nine-story building subject to ground acceleration $\ddot{x}_g$, as shown in Fig. 6. The external force is expressed as

$$\mathbf{f}(t) = -\mathbf{M}\mathbf{\Gamma}\ddot{x}_g(t) \quad (23)$$

where $\mathbf{\Gamma}$ is the load vector that captures inertial forces due to the ground acceleration $\ddot{x}_g$. Material properties, geometry, initial conditions, and boundary conditions are identical to the nine-story building north-south moment-resisting frame in (Ohtori et al. 2004). Each beam and column is discretized with ten 2-noded Euler-Bernoulli frame elements and the HRZ lumping scheme (Hinton et al. 1976) is used for the mass matrix, so that the first ten natural frequencies are 0.443, 1.17, 2.02, 2.85, 3.12, 3.57, 3.69, 3.84, 3.89, and 4.03 Hz. The structure is assumed to have mass- and stiffness-proportional Rayleigh damping such that the first and the tenth modal damping ratios are both 0.02. For domain decomposition, we partition the building structure into two subdomains, as shown in Fig. 6. We compare the numerical solutions obtained using the UTS method with $\Delta t = 1/60$ sec, and the VC-MTS and proposed ML-MTS methods with $\Delta t^{(2)} = 1/60$ sec. For time integration, we again use the implicit Newmark scheme with $\gamma^{(s)} = 0.5$ and $\beta^{(s)} = 0.25$ (i.e.

trapezoidal rule).

For this problem, we train the deep LSTM network using the same network architecture, data normalization, training-validation split ratio, optimizer, loss function, and hyperparameters as in the previous section. A total of 800 training data and 200 validation data are obtained from numerical solutions using the VC-MTS method with $\Delta t^{(1)} = \Delta t^{(2)} = 1/60$ sec over a time duration of $T = 10$ sec. Each input sequence consists of random white Gaussian noise samples with a sampling frequency of 50 Hz. Time taken for generating these data and training the LSTM model on our hardware are 953 sec and 6,251 sec, respectively. These costs can be lowered by training on better hardware.

To test the performance of the proposed ML-MTS method, we consider one chirp signal $\ddot{x}_g(t) = 0.1 \sin(2\pi t^2/T)$ and two earthquake records: (i) El Centro: the north-south component recorded at the Imperial Valley Irrigation District substation in El Centro, California, during the Imperial Valley, California earthquake of May 18, 1940; and (ii) Kobe: the north-south component recorded at the Kobe Japanese Meterological Agency station, during the Hyogoken Nanbu earthquake of January 17, 1995. To maintain linear elastic response of the structure, the earthquake records are scaled by a factor of 0.4. The ground acceleration is sampled at 0.02 sec intervals and for time steps that do not correspond to discrete sampling points, a shape-preserving piecewise cubic interpolation is used to obtain the ground accelerations at those instants.

Figs. 7 and 8 compare the time history of Lagrange multipliers at the receiver for the VC-MTS and the proposed ML-MTS methods. The reference solution is obtained using the VC-MTS method with a small time step of $\Delta t^{(1)} = \Delta t^{(2)} = 1/600$ sec. The results show that the deep LSTM network accurately predicts the Lagrange multipliers. Figs. 9 and 10 show the horizontal and vertical displacements at the receiver and the corresponding local instantaneous errors for the UTS, VC-MTS, and ML-MTS methods. We take the numerical solution using the UTS method with a small time step of $1/60000$ sec as the reference solution. The computational costs, errors, and drift in displacements for the UTS, VC-MTS, and the proposed ML-MTS methods are summarized in Table 1. We observe that the proposed ML-MTS method achieves about 2x over the conventional

VC-MTS method, while also resulting in smaller errors. Compared to conventional UTS time integration scheme (i.e. Newmark Trapezoidal method), the proposed ML-MTS method achieves an even more impressive 4x speedup.

However, if one accounts for the cost of synthetic data generation and training as well, then these costs would need to be amortized over numerous simulations to realize the benefit of ML-MTS approach. For instance, let the cost of generating one offline data-set be C_{data} and the cost of training the deep LSTM network be C_{train} . Let the cost of one simulation for the ML-MTS method be C_{ML-MTS} and that for the MTS method be C_{MTS} . Finally, let number of simulations to run be N_{sim} . Then, the total cost, including online and offline costs for the proposed ML-MTS method is computed to be: $N_{sim}C_{ML-MTS} + C_{data} + C_{train}$ whereas that of the MTS method is: $N_{sim}C_{MTS}$. Thus, the minimum number of simulations needed to realize the computational gains of the ML-MTS method is:

$$N_{sim} = \frac{C_{data} + C_{train}}{C_{MTS} - C_{ML-MTS}} \quad (24)$$

For the nine-story building problem, N_{sim} is found to be about 8,000 simulations similar to the El Centro excitation. Better hardware and pipeline parallelism can be leveraged to reduce this number. However, as with all ML applications, there is a large initial cost associated with the ML-MTS method.

ERROR ANALYSIS

In this section, we evaluate the accuracy of the proposed ML-MTS method by carrying out a truncation error analysis, investigating error propagation from the predicted Lagrange multiplier to subdomain solutions. Furthermore, since the proposed ML-MTS approach enforces continuity of all kinematic quantities, errors from one subdomain can propagate to adjacent subdomains. Thus, we consider two main perturbed quantities – Lagrange multipliers and boundary / interface quantities – to investigate the subdomain solution accuracy.

Let the problem domain be decomposed into S subdomains $\Omega^{(1)}, \Omega^{(2)}, \dots, \Omega^{(S)}$ where subdomain $\Omega^{(s)}$ is integrated with a time-stepping scheme with an order of accuracy of $k^{(s)}$. We compute

the local truncation errors (LTEs) in the solution obtained by the proposed ML-MTS method by advancing from t_n to t_{n+1} . First, we define the following LTEs in velocity and displacement for subdomain $\Omega^{(s)}$:

$$\mathbf{e}_v^{(s)} = \mathbf{v}_{n+1}^{(s)} - \mathbf{v}^{(s)}(t_{n+1}^{(s)}), \quad \mathbf{e}_d^{(s)} = \mathbf{d}_{n+1}^{(s)} - \mathbf{d}^{(s)}(t_{n+1}^{(s)}) \quad (25)$$

where $\mathbf{v}^{(s)}(t_{n+1}^{(s)})$ and $\mathbf{d}^{(s)}(t_{n+1}^{(s)})$ denote the exact velocity and displacement, respectively, at time $t_{n+1}^{(s)}$. Assuming that the kinematic quantities at t_n are exact (i.e. $\mathbf{v}^{(s)}(t_n) = \mathbf{v}_n^{(s)}$ and $\mathbf{d}^{(s)}(t_n) = \mathbf{d}_n^{(s)}$), their values at $t_{n+1}^{(s)}$ are obtained using a Taylor series expansion about $t = t_n$:

$$\mathbf{v}^{(s)}(t_{n+1}^{(s)}) = \sum_{k=0}^{\infty} \frac{(\Delta T)^k}{k!} \left. \frac{d^k \mathbf{v}^{(s)}}{dt^k} \right|_{t=t_n}, \quad \mathbf{d}^{(s)}(t_{n+1}^{(s)}) = \sum_{k=0}^{\infty} \frac{(\Delta T)^k}{k!} \left. \frac{d^k \mathbf{d}^{(s)}}{dt^k} \right|_{t=t_n} \quad (26)$$

From Eq. (10), the LTEs can be written as:

$$\mathbf{e}_v^{(s)} = \underbrace{\bar{\mathbf{v}}_{n+1}^{(s)} - \bar{\mathbf{v}}_{n+1}^{(s)}(t_{n+1}^{(s)})}_{\bar{\mathbf{e}}_v^{(s)}} - \underbrace{\mathbf{Y}_v^{(s)}(\lambda_{n+1} - \lambda(t_{n+1}))}_{\delta \lambda}, \quad \mathbf{e}_d^{(s)} = \underbrace{\bar{\mathbf{d}}_{n+1}^{(s)} - \bar{\mathbf{d}}_{n+1}^{(s)}(t_{n+1}^{(s)})}_{\bar{\mathbf{e}}_d^{(s)}} - \underbrace{\mathbf{Y}_d^{(s)}(\lambda_{n+1} - \lambda(t_{n+1}))}_{\delta \lambda} \quad (27)$$

where $\mathbf{Y}_v^{(s)} \propto O(\Delta T)$ and $\mathbf{Y}_d^{(s)} \propto O(\Delta T^2)$ (Kwon and Prakash 2025a) and $\delta \lambda$ denotes perturbed Lagrange multipliers. Note that $\bar{\mathbf{e}}_v^{(s)}$ and $\bar{\mathbf{e}}_d^{(s)}$ represent the LTEs in velocity and displacement, respectively, of the underlying time integration scheme in subdomain $\Omega^{(s)}$.

Due to the connectivity of subdomain $\Omega^{(s)}$ with its neighboring subdomains $\Omega^{(q)}$ across interface $\Gamma^{(s,q)}$ over which all kinematics quantities are prescribed as boundary conditions, errors from subdomains $\Omega^{(q)}$ propagate to subdomain $\Omega^{(s)}$:

$$\delta \mathbf{v}^{(s)} \propto O(\Delta T^{k^{(q)}+1}), \quad \delta \mathbf{d}^{(s)} \propto O(\Delta T^{k^{(q)}+1}) \quad \text{on } \Gamma^{(s,q)} \quad (28)$$

where $k^{(q)}$ is the order of accuracy of the adjacent subdomains $\Omega^{(q)}$. Thus, the LTEs of the underlying time integration scheme are:

$$\bar{\mathbf{e}}_v^{(s)} \propto O(\Delta T^{\min(k^{(s)}+1, k^{(q)}+1)}), \quad \bar{\mathbf{e}}_d^{(s)} \propto O(\Delta T^{\min(k^{(s)}+1, k^{(q)}+1)}) \quad (29)$$

Thus, order of accuracy of the underlying time integration scheme within subdomain $\Omega^{(s)}$ is the lower of its own time integration scheme and that of its neighbors which impose boundary conditions on subdomain $\Omega^{(s)}$.

Next, let the order of perturbations in the Lagrange multipliers obtained from the ML model be k^λ (i.e. $\delta\lambda = O(\Delta T^{k^\lambda})$). From Eq. (27), one obtains

$$\begin{bmatrix} \mathbf{e}_v^{(s)} \\ \mathbf{e}_d^{(s)} \end{bmatrix} \propto \begin{bmatrix} O(\Delta T^{\min(k^{(s)}, k^{(q)})+1}) \\ O(\Delta T^{\min(k^{(s)}, k^{(q)})+1}) \end{bmatrix} - \begin{bmatrix} O(\Delta t) \\ O(\Delta t^2) \end{bmatrix} O(\Delta T^{k^\lambda}) \propto \begin{bmatrix} O(\Delta T^{\min(k^{(s)}, k^{(q)}, k^\lambda)+1}) \\ O(\Delta T^{\min(k^{(s)}, k^{(q)}, k^\lambda)+1}) \end{bmatrix} \quad (30)$$

Since the lowest order of LTE among $\mathbf{e}_v^{(s)}$ and $\mathbf{e}_d^{(s)}$ is $O(\Delta T^{\min(k^{(s)}, k^{(q)}, k^\lambda)+1})$, the overall rate of convergence of the proposed ML-MTS method in subdomain $\Omega^{(s)}$ is $\min(k^{(s)}, k^{(q)}, k^\lambda)$.

To verify the theoretical LTEs obtained above, we compute the convergence rates for the 3-DOF problem described in the previous section. This problem is solved using the proposed ML-MTS method, where the Lagrange multipliers are obtained by adding a perturbation to the exact ones, instead of by using trained ML model. The external load is taken as $f_1(t) = \sin(\pi t)$ over $T = 10$ with $\Delta t^{(1)} = 2\Delta t^{(2)} = 5\Delta t^{(3)}$. For the underlying time integration scheme, we use the implicit Newmark scheme with two parameter sets: 1) $\gamma^{(s)} = 0.5$ and $\beta^{(s)} = 0.25$ (i.e. trapezoidal rule), which gives second-order accuracy, $k^{(s)} = 2$; and 2) $\gamma^{(s)} = 0.6$ and $\beta^{(s)} = 0.3025$, which results in first-order accuracy, $k^{(s)} = 1$. Fig. 11 shows average simulation errors for different choices of time integration parameters and for various perturbation levels in Lagrange multipliers. We observe that the rates of convergence in $\Omega^{(1)}$, $\Omega^{(2)}$, and $\Omega^{(3)}$ are, respectively, $\min(k^{(1)}, k^{(2)}, k^{(3)}, k^\lambda)$, $\min(k^{(2)}, k^{(3)}, k^\lambda)$, and $\min(k^{(3)}, k^\lambda)$. These results are consistent with the theoretical analysis, reflecting the order of solution of subdomains $\Omega^{(3)} \rightarrow \Omega^{(2)} \rightarrow \Omega^{(1)}$.

CONCLUDING REMARKS

We propose a novel ML-MTS method to accelerate multi-scale simulations in structural dynamics by eliminating the time-consuming 3-step process involved with conventional MTS methods. The proposed ML-MTS method leverages a deep LSTM network to predict the time series of

Lagrange multipliers on the interface between subdomains. The predicted Lagrange multipliers are used as external forces, while the known solutions from adjacent subdomains are enforced as prescribed boundary conditions. Thus, the proposed method enables coupling of subdomains in a single pass while imposing continuity of all kinematic quantities, allowing the use of multiple time integration schemes and different time steps.

To assess the performance of the proposed ML-MTS methods, we simulate a 3-DOF problem with three subdomains and a nine-story building subject to two earthquakes. Results show that the Lagrange multipliers predicted using the deep LSTM network are accurate in comparison to those obtained using conventional MTS methods. Further, while existing VC-MTS methods can only enforce the continuity of velocities between subdomains, the proposed ML-MTS method is able to eliminate the drift in displacements by enforcing the continuity of all kinematic quantities. The numerical examples presented demonstrate that the proposed ML-MTS method yields accurate solutions at low computational cost, compared to existing MTS and UTS methods in the literature.

While the proposed ML-MTS method performs well for the numerical examples presented herein, one drawback of this approach is that the training of the LSTM network is problem dependent and sensitive to the choice of hyperparameters. The formulation is also currently limited to linear problems. Further research is needed to explore alternatives of the LSTM network that can overcome these limitations and to extend the approach to nonlinear problems. Additionally, while in principle the proposed ML-MTS can be applied to large-scale problems with any number of subdomains, its performance in terms of stability, accuracy, and robustness needs to be investigated for high-dimensional large-scale problems.

DATA AVAILABILITY STATEMENT

All data, models, and code for the study are available at https://github.com/kwon193/MTS_ML.git.

ACKNOWLEDGMENTS

This work is partly supported by the National Science Foundation (NSF) through Grant CNS-2229136: Collaborative Research: CPS: Co-Designed Control and Scheduling Adaptation for

Assured Cyber-Physical System Safety and Performance. The authors also acknowledge support from the Computational Interdisciplinary Graduate Program (CIGP) of Purdue University. This work was supported by 2026 Hongik University Research Fund.

REFERENCES

- Abedi, R., Hawker, M. A., Haber, R. B., and Matouš, K. (2010). “An adaptive spacetime discontinuous galerkin method for cohesive models of elastodynamic fracture.” *International journal for numerical methods in engineering*, 81(10), 1207–1241.
- Bathe, K.-J. (2016). *Finite element procedures. 2nd ed.* Higher Education Press China, Watertown (MA).
- Bathe, K.-J. and Noh, G. (2012). “Insight into an implicit time integration scheme for structural dynamics.” *Computers & Structures*, 98, 1–6.
- Belytschko, T., Liu, W. K., and Smolinski, P. (1984). “Multi-stepping implicit-explicit procedures in transient analysis.” *Innovative methods in Nonlinear Analysis*, Pineridge Press, 135–154.
- Belytschko, T. and Mullen, R. (1976). “Mesh partitions of explicit-implicit time integration.” *Formulations and computational algorithms in finite element analysis*, 673–690.
- Belytschko, T. and Mullen, R. (1978). “Stability of explicit-implicit mesh partitions in time integration.” *International Journal for Numerical Methods in Engineering*, 12(10), 1575–1586.
- Belytschko, T., Yen, H.-J., and Mullen, R. (1979). “Mixed methods for time integration.” *Computer Methods in Applied Mechanics and Engineering*, 17–18, 259–275.
- Bottasso, C. L. (2002). “Multiscale temporal integration.” *Computer Methods in Applied Mechanics and Engineering*, 191(25-26), 2815–2830.
- Brun, M., Gravouil, A., Combescure, A., and Limam, A. (2015). “Two FETI-based heterogeneous time step coupling methods for Newmark and α -schemes derived from the energy method.” *Computer Methods in Applied Mechanics and Engineering*, 283, 130–176.
- Bursi, O. S., He, L., Bonelli, A., and Pegon, P. (2010). “Novel generalized- α methods for interfield parallel integration of heterogeneous structural dynamic systems.” *Journal of Computational and Applied Mathematics*, 234(7), 2250–2258.

- Chiuso, A. and Pillonetto, G. (2019). “System identification: A machine learning perspective.” *Annual Review of Control, Robotics, and Autonomous Systems*, 2(1), 281–304.
- Chopra, A. K. (2012). *Dynamics of structures: theory and applications to earthquake engineering*. Pearson Education.
- Combescure, A. and Gravouil, A. (2002). “A numerical scheme to couple subdomains with different time-steps for predominantly linear transient analysis.” *Computer methods in applied mechanics and engineering*, 191(11-12), 1129–1157.
- Courant, R., Friedrichs, K., and Lewy, H. (1928). “Über die partiellen differenzengleichungen der mathematischen physik.” *Mathematische Annalen*, 100(1), 32–74.
- Crouch, R. and Oskay, C. (2015). “Accelerated time integrator for multiple time scale homogenization.” *International Journal for Numerical Methods in Engineering*, 101(13), 1019–1042.
- Cunha, B. Z., Droz, C., Zine, A.-M., Foulard, S., and Ichchou, M. (2023). “A review of machine learning methods applied to structural dynamics and vibroacoustic.” *Mechanical Systems and Signal Processing*, 200, 110535.
- Daniel, W. (1997). “The subcycled Newmark algorithm.” *Computational Mechanics*, 20(3), 272–281.
- Daniel, W. J. T. (1998). “A study of the stability of subcycling algorithms in structural dynamics.” *Computer methods in applied mechanics and engineering*, 156(1-4), 1–13.
- Farhat, C., Crivelli, L., and Roux, F.-X. (1994). “A transient FETI methodology for large-scale parallel implicit computations in structural mechanics.” *International Journal for Numerical Methods in Engineering*, 37(11), 1945–1975.
- Farhat, C. and Roux, F.-X. (1991). “A method of finite element tearing and interconnecting and its parallel solution algorithm.” *International journal for numerical methods in engineering*, 32(6), 1205–1227.
- Farhat, C. and Roux, F.-X. (1994). “Implicit parallel processing in structural mechanics.” *Comp. Mech. Advances*, 2, 1–124.
- Farrar, C. R. and Worden, K. (2012). *Structural health monitoring: a machine learning perspective*.

John Wiley & Sons.

- Gates, M., Matouš, K., and Heath, M. T. (2008). “Asynchronous multi-domain variational integrators for non-linear problems.” *International journal for numerical methods in engineering*, 76(9), 1353–1378.
- Gravouil, A. and Combescure, A. (2001). “Multi-time-step explicit–implicit method for non-linear structural dynamics.” *International Journal for Numerical Methods in Engineering*, 50(1), 199–225.
- Gravouil, A., Combescure, A., and Brun, M. (2015). “Heterogeneous asynchronous time integrators for computational structural dynamics.” *International Journal for Numerical Methods in Engineering*, 102(3-4), 202–232.
- Guarize, R., Matos, N., Sagrilo, L., and Lima, E. (2007). “Neural networks in the dynamic response analysis of slender marine structures.” *Applied Ocean Research*, 29(4), 191–198.
- Hinton, E., Rock, T., and Zienkiewicz, O. (1976). “A note on mass lumping and related processes in the finite element method.” *Earthquake Engineering & Structural Dynamics*, 4(3), 245–249.
- Hughes, T. J. (2000). *The finite element method: linear static and dynamic finite element analysis*. Dover Publications.
- Hughes, T. J. and Liu, W. (1978a). “Implicit-explicit finite elements in transient analysis: implementation and numerical examples.” *Journal of Applied Mechanics*, 45(2), 375–378.
- Hughes, T. J. and Liu, W. (1978b). “Implicit-explicit finite elements in transient analysis: stability theory.” *Journal of Applied Mechanics*, 45(2), 371–374.
- Hulbert, G. M. and Hughes, T. J. R. (1990). “Space-time finite element methods for second-order hyperbolic equations.” *Computer Methods in Applied Mechanics and Engineering*, 84, 327–348.
- Karimi, S. and Nakshatrala, K. (2014). “On multi-time-step monolithic coupling algorithms for elastodynamics.” *Journal of Computational Physics*, 273, 671–705.
- Kingma, D. P. and Ba, J. (2014). “Adam: A method for stochastic optimization.” *arXiv preprint arXiv:1412.6980*.
- Kwon, S.-B. and Prakash, A. (2025a). “Coupling composite schemes with different time steps for

multi-scale structural dynamics.” *International Journal for Multiscale Computational Engineering*, 23(3), 63–109.

Kwon, S.-B. and Prakash, A. (2025b). “A unified computational framework for developing general multi-time-step (MTS) coupling schemes for time-dependent problems.” *Engineering with Computers*, 41(6), 4727–4756.

Lagaros, N. D. and Papadrakakis, M. (2012). “Neural network based prediction schemes of the non-linear seismic response of 3d buildings.” *Advances in Engineering Software*, 44(1), 92–115.

LeCun, Y., Bengio, Y., et al. (1995). “Convolutional networks for images, speech, and time series.” *The handbook of brain theory and neural networks*, 3361(10), 1995.

Lee, I.-T. A., Leiserson, C. E., Schardl, T. B., Zhang, Z., and Sukha, J. (2015). “On-the-fly pipeline parallelism.” *ACM Transactions on Parallel Computing (TOPC)*, 2(3), 1–42.

Lew, A., Marsden, J. E., Ortiz, M., and West, M. (2003). “Asynchronous variational integrators.” *Archive for Rational Mechanics and Analysis*, 167(2), 85–146.

Liu, W. K. and Belytschko, T. (1982). “Mixed-time implicit-explicit finite elements for transient analysis.” *Computers & Structures*, 15(4), 445–450.

Mahjoubi, N., Gravouil, A., Combescure, A., and Greffet, N. (2011). “A monolithic energy conserving method to couple heterogeneous time integrators with incompatible time steps in structural dynamics.” *Computer Methods in Applied Mechanics and Engineering*, 200(9-12), 1069–1086.

Mahjoubi, N. and Krenk, S. (2010). “Multi-time-step domain coupling method with energy control.” *International Journal for Numerical Methods in Engineering*, 83(13), 1700–1718.

Miao, J., Zhu, P., Shi, G., and Chen, G. (2008). “Study on sub-cycling algorithm for flexible multi-body system—integral theory and implementation flow chart.” *Computational Mechanics*, 41(2), 257–268.

Newmark, N. M. (1959). “A method of computation for structural dynamics.” *Journal of Engineering Mechanics*, 85(3), 67–94.

Ohtori, Y., Christenson, R., Spencer Jr, B., and Dyke, S. (2004). “Benchmark control problems for seismically excited nonlinear buildings.” *Journal of Engineering Mechanics*, 130(4), 366–385.

- Park, K. (1980). "Partitioned transient analysis procedures for coupled-field problems: stability analysis." *Journal of Applied Mechanics*, 47(2), 370–376.
- Prakash, A. and Hjelmstad, K. D. (2004). "A FETI-based multi-time-step coupling method for Newmark schemes in structural dynamics." *International Journal for Numerical Methods in Engineering*, 61(13), 2183–2204.
- Prakash, A., Taciroglu, E., and Hjelmstad, K. D. (2014). "Computationally efficient multi-time-step method for partitioned time integration of highly nonlinear structural dynamics." *Computers & Structures*, 133, 51–63.
- Seibold, A. S. and Rixen, D. J. (2019). "Asynchronous time integration in structural mechanics." *PAMM*, 19(1), e201900495.
- Smolinski, P. (1992). "An explicit multi-time step integration method for second order equations." *Computer methods in applied mechanics and engineering*, 94(1), 25–34.
- Smolinski, P., Belytschko, T., and Neal, M. (1988). "Multi-time-step integration using nodal partitioning." *International Journal for Numerical Methods in Engineering*, 26(2), 349–359.
- Subber, W. and Matouš, K. (2016). "Asynchronous space–time algorithm based on a domain decomposition method for structural dynamics problems on non-matching meshes." *Computational Mechanics*, 57(2), 211–235.
- Waisman, H. and Fish, J. (2008). "A heterogeneous space–time full approximation storage multilevel method for molecular dynamics simulations." *International journal for numerical methods in engineering*, 73(3), 407–426.
- Wiberg, N.-E. and Li, X. (1999). "Adaptive finite element procedures for linear and non-linear dynamics." *International Journal for Numerical Methods in Engineering*, 46, 1781–1802.
- Williams, R. J. and Zipser, D. (1989). "A learning algorithm for continually running fully recurrent neural networks." *Neural computation*, 1(2), 270–280.
- Xiang, P., Zhang, P., Zhao, H., Shao, Z., and Jiang, L. (2024). "Seismic response prediction of a train-bridge coupled system based on a lstm neural network." *Mechanics Based Design of Structures and Machines*, 52(8), 5673–5695.

- 578 Zhang, R., Chen, Z., Chen, S., Zheng, J., Büyüköztürk, O., and Sun, H. (2019). “Deep long
579 short-term memory networks for nonlinear structural seismic response prediction.” *Computers
580 & Structures*, 220, 55–68.
- 581 Zhang, R., Liu, Y., and Sun, H. (2020a). “Physics-guided convolutional neural network (PhyCNN)
582 for data-driven seismic response modeling.” *Engineering Structures*, 215, 110704.
- 583 Zhang, R., Liu, Y., and Sun, H. (2020b). “Physics-informed multi-lstm networks for metamodel-
584 ing of nonlinear structures.” *Computer Methods in Applied Mechanics and Engineering*, 369,
585 113226.
- 586 Zienkiewicz, O. C. and Taylor, R. L. (2005). *The finite element method for solid and structural
587 mechanics*. Elsevier.

588

List of Tables

589

1 Computational costs, average simulation errors, and maximum drift for different

590

numerical methods in the nine-story building problem. 27

TABLE 1. Computational costs, average simulation errors, and maximum drift for different numerical methods in the nine-story building problem.

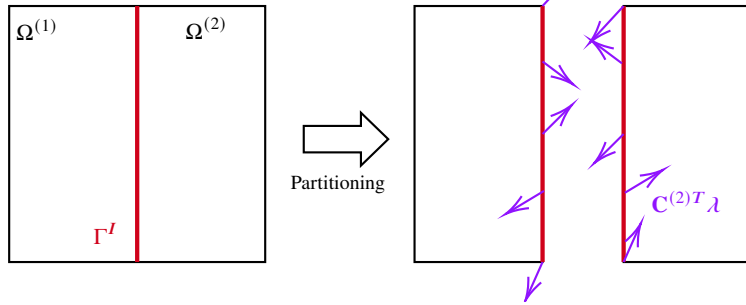
Earthquake/ Excitation	Method	Computational cost (sec)					Average errors (%)			Drift (mm)
		LSTM	Subdomain	λ	Update	Total	$E(\lambda)$	$E(\mathbf{d})$	$E(\mathbf{v})$	
Chirp ($\Delta t^{(1)} = 10\Delta t^{(2)}$)	UTS	-	-	-	-	3.49	-	0.04	0.07	-
	VC-MTS	-	0.60	0.01	0.60	1.22	3.78	1.06	1.37	1.20
	VCDC-MTS	-	0.58	0.02	0.60	1.21	4.28	1.19	1.50	0.00
	Proposed ML-MTS	0.03	-	-	0.50	0.53	0.54	0.34	0.77	0.00
Chirp ($\Delta t^{(1)} = 6\Delta t^{(2)}$)	UTS	-	-	-	-	3.49	-	0.04	0.07	-
	VC-MTS	-	0.77	0.02	0.77	1.56	1.95	0.59	1.05	0.51
	VCDC-MTS	-	0.76	0.03	0.78	1.57	2.16	0.64	1.10	0.00
	Proposed ML-MTS	0.03	-	-	0.62	0.65	0.54	0.23	0.40	0.00
El Centro ($\Delta t^{(1)} = 6\Delta t^{(2)}$)	UTS	-	-	-	-	3.33	-	0.12	0.21	-
	VC-MTS	-	0.81	0.02	0.80	1.63	3.68	1.55	1.83	2.50
	VCDC-MTS	-	0.82	0.02	0.81	1.65	3.56	1.50	1.82	0.00
	Proposed ML-MTS	0.03	-	-	0.73	0.75	0.56	0.47	0.76	0.00
Kobe ($\Delta t^{(1)} = 6\Delta t^{(2)}$)	UTS	-	-	-	-	4.01	-	0.12	0.21	-
	VC-MTS	-	0.92	0.02	0.92	1.86	3.04	1.15	1.58	5.73
	VCDC-MTS	-	1.16	0.03	1.16	2.35	3.18	1.20	1.59	0.00
	Proposed ML-MTS	0.03	-	-	0.92	0.95	1.09	0.59	0.82	0.00

List of Figures

- 1 Domain decomposition into two subdomains. In each subdomain $\Omega^{(s)}$, different time steps $\Delta t^{(s)}$ are used to advance subdomain solutions. A block time step ΔT is used to advance the coupled system. 30
- 2 Architecture of a deep LSTM network. (a) Input $\ddot{x}_{g,n}$ is the ground motion at t_n and ℓ_n^l and h_n^l denote the output and hidden states, respectively, of the l th layer at t_n . (b) Schematic of a LSTM cell with a forget gate 31
- 3 Computation sequence of subdomains to advance the solution from t_n to t_{n+1} . Solved subdomains are denoted as Ω_{solved} and subdomains adjacent to Ω_{solved} are designated as $\Omega_{neighbor}$. Once subdomain $\Omega^{(s)}$ is solved, it is moved from $\Omega_{neighbor}$ to Ω_{solved} and the set $\Omega_{neighbor}$ is updated to identify the next subdomain to solve. 32
- 4 3-DOF system and its decomposition into three subdomains. 33
- 5 Results for the 3-DOF problem: (a)-(b) time history and errors in Lagrange multipliers, (c)-(f) Displacement time history and errors for Nodes 2 and 3. Displacement drifts $D(d_2)$ at Node 2 for the VC-MTS and vanilla ML-MTS methods are found to be 3.45×10^{-5} and 0.74, respectively. Displacement drifts $D(d_3)$ at Node 3 for the VC-MTS and vanilla ML-MTS methods are found to be 4.56×10^{-6} and 0.75×10^{-2} , respectively. The proposed ML-MTS method does not exhibit any displacement drift. 34
- 6 A nine-story building subject to earthquake ground motion solved by decomposing into two subdomains. 35
- 7 Time history of Lagrange multipliers at the receiver in the nine-story building problem for the chirp signal. 36
- 8 Time history of Lagrange multipliers at the receiver in the nine-story building problem for two earthquake records with $\Delta t^{(1)} = 6\Delta t^{(2)}$ 37

616	9	Time history of horizontal and vertical displacements at the receiver in the nine-	
617		story building problem for the chirp signal. The upper curves represent displace-	
618		ments and the lower curves represent relative errors.	38
619	10	Time history of horizontal and vertical displacements at the receiver in the nine-	
620		story building problem for two earthquake records with $\Delta t^{(1)} = 6\Delta t^{(2)}$. The upper	
621		curves represent displacements and the lower curves represent relative errors. . . .	39
622	11	Average simulation errors in velocities and displacements in the 3-DOF problem. .	40

Space



Time

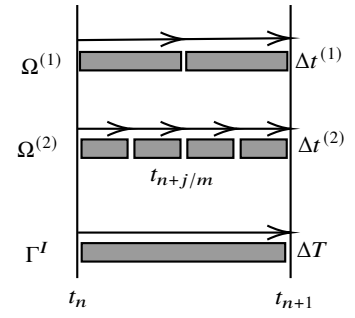
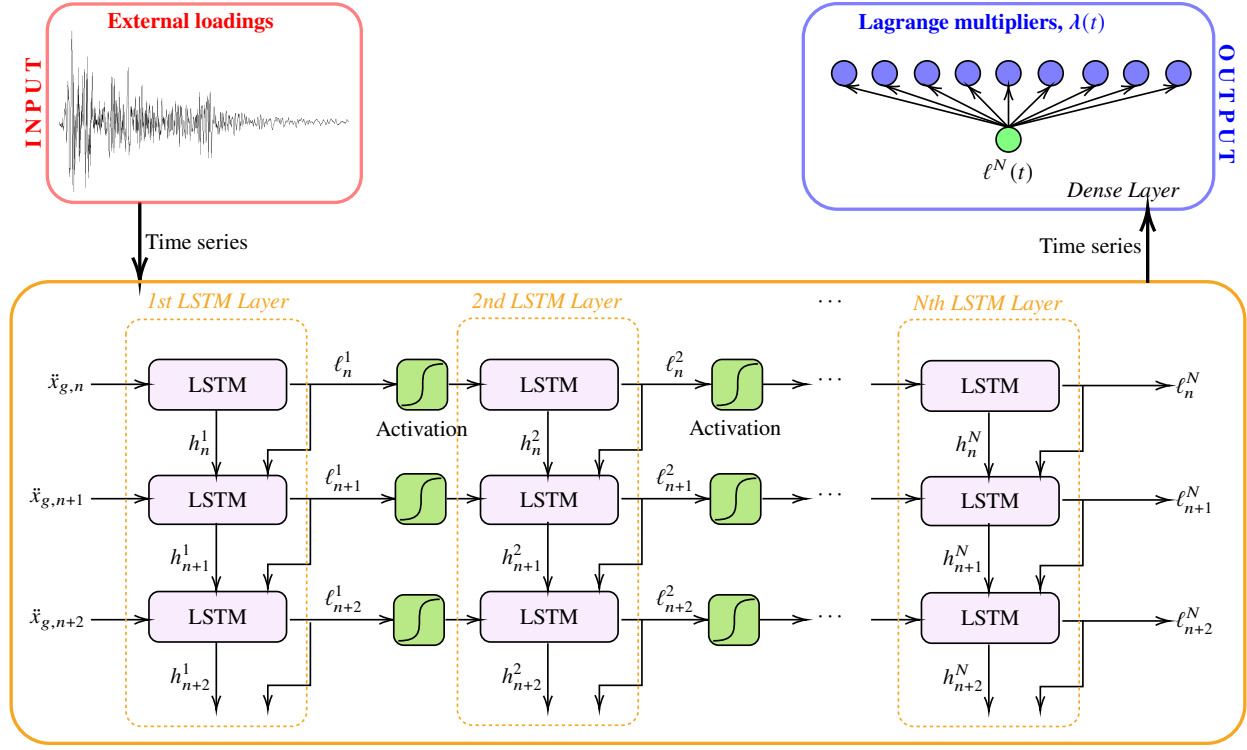


Fig. 1. Domain decomposition into two subdomains. In each subdomain $\Omega^{(s)}$, different time steps $\Delta t^{(s)}$ are used to advance subdomain solutions. A block time step ΔT is used to advance the coupled system.



(a)

(b)

Fig. 2. Architecture of a deep LSTM network. (a) Input $\ddot{x}_{g,n}$ is the ground motion at t_n and ℓ_n^l and h_n^l denote the output and hidden states, respectively, of the l th layer at t_n . (b) Schematic of a LSTM cell with a forget gate

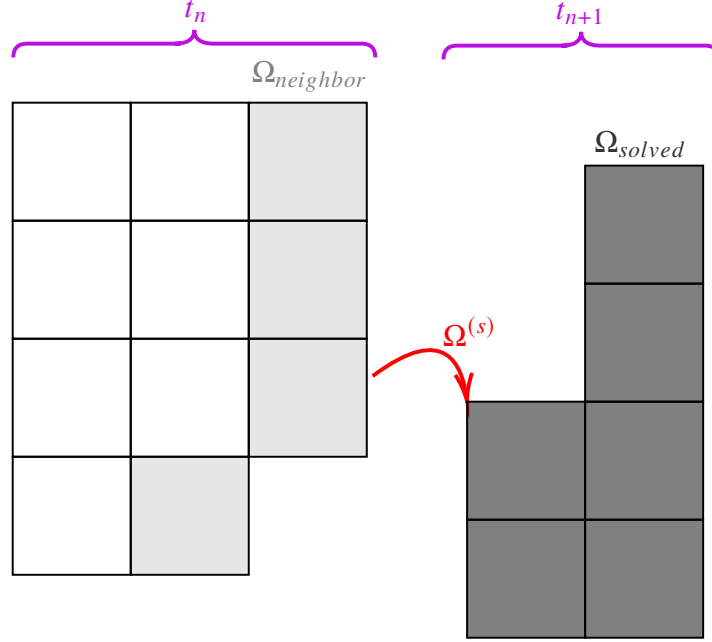


Fig. 3. Computation sequence of subdomains to advance the solution from t_n to t_{n+1} . Solved subdomains are denoted as Ω_{solved} and subdomains adjacent to Ω_{solved} are designated as $\Omega_{neighbor}$. Once subdomain $\Omega^{(s)}$ is solved, it is moved from $\Omega_{neighbor}$ to Ω_{solved} and the set $\Omega_{neighbor}$ is updated to identify the next subdomain to solve.

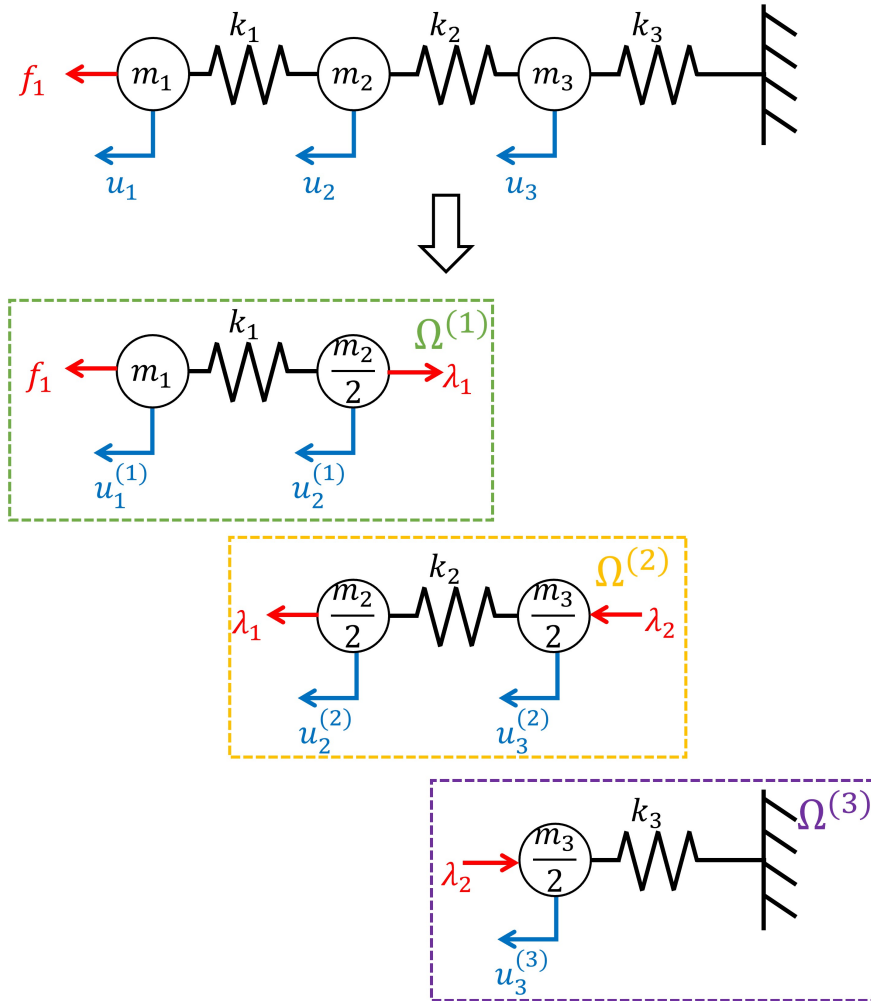


Fig. 4. 3-DOF system and its decomposition into three subdomains.

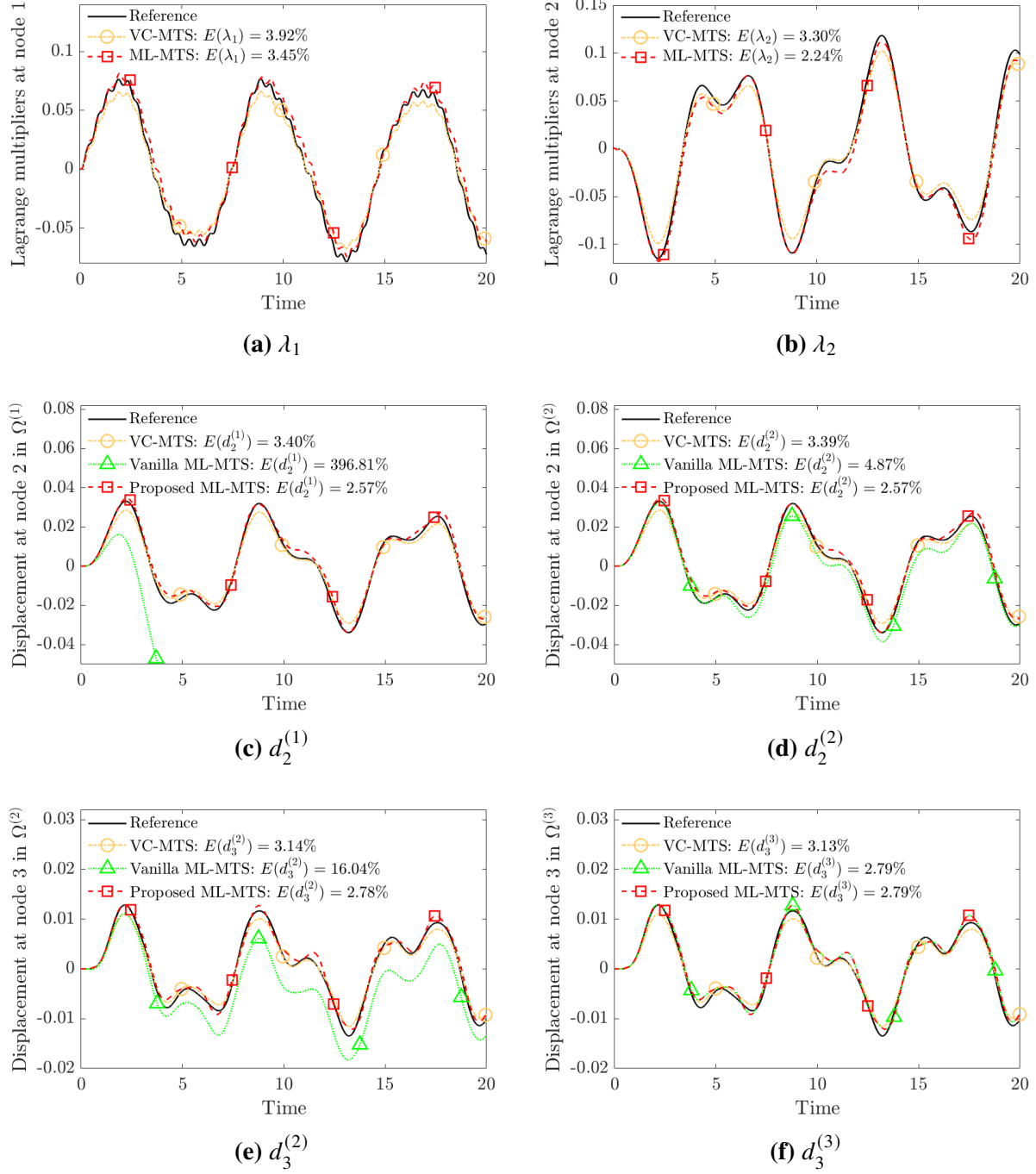


Fig. 5. Results for the 3-DOF problem: (a)-(b) time history and errors in Lagrange multipliers, (c)-(f) Displacement time history and errors for Nodes 2 and 3. Displacement drifts $D(d_2)$ at Node 2 for the VC-MTS and vanilla ML-MTS methods are found to be 3.45×10^{-5} and 0.74 , respectively. Displacement drifts $D(d_3)$ at Node 3 for the VC-MTS and vanilla ML-MTS methods are found to be 4.56×10^{-6} and 0.75×10^{-2} , respectively. The proposed ML-MTS method does not exhibit any displacement drift.

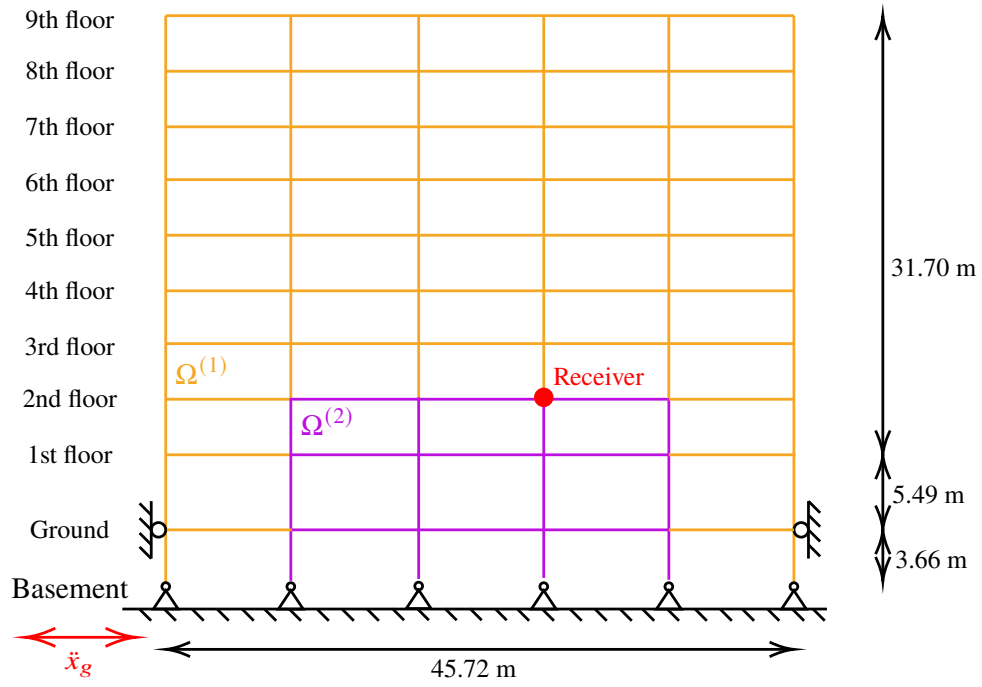
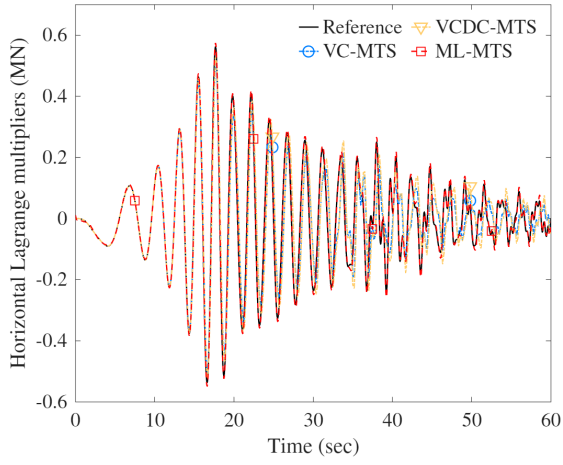
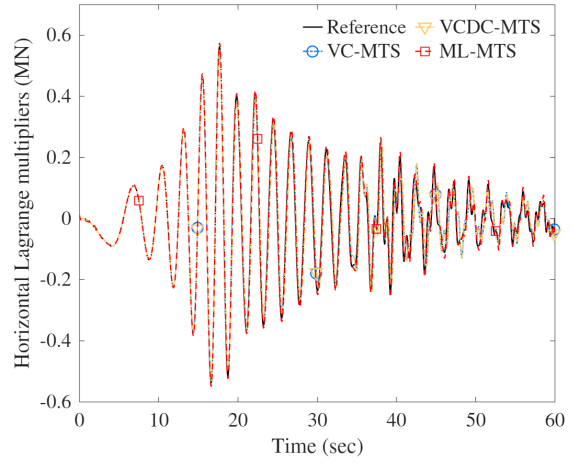


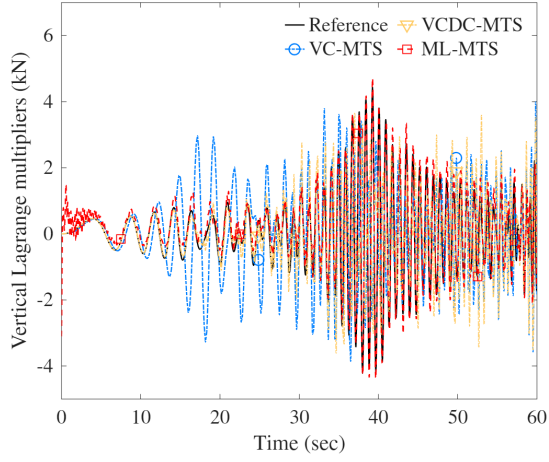
Fig. 6. A nine-story building subject to earthquake ground motion solved by decomposing into two subdomains.



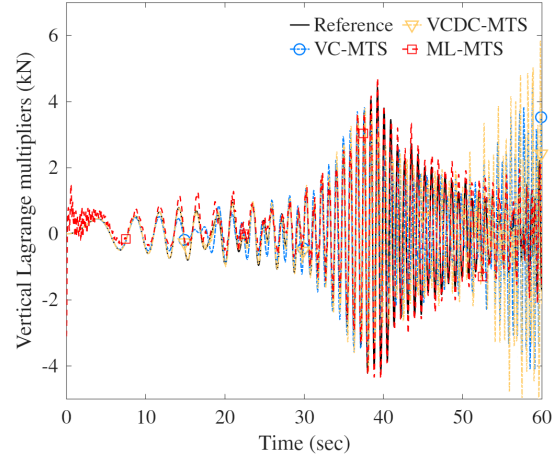
(a) Horizontal DOF when $\Delta t^{(1)} = 10\Delta t^{(2)}$



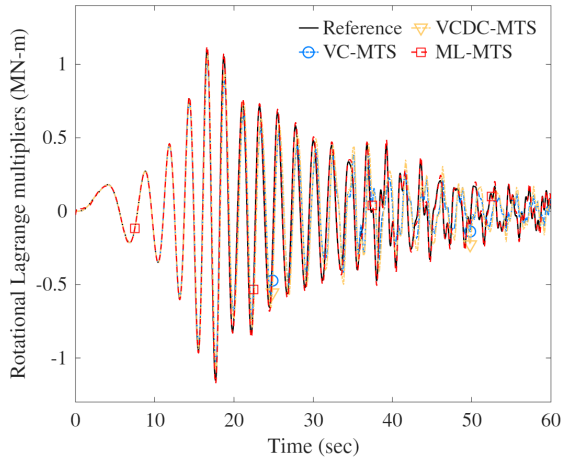
(b) Horizontal DOF when $\Delta t^{(1)} = 6\Delta t^{(2)}$



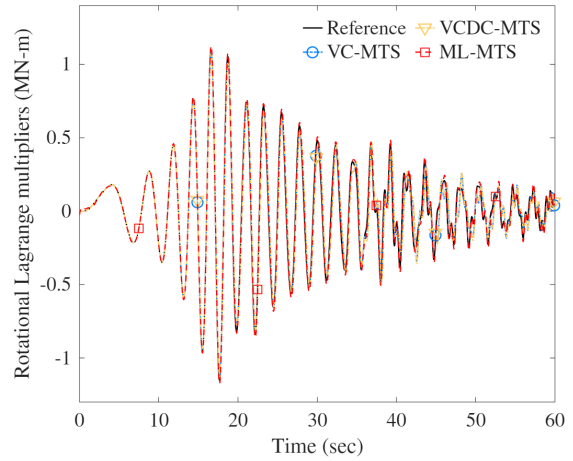
(c) Vertical DOF when $\Delta t^{(1)} = 10\Delta t^{(2)}$



(d) Vertical DOF when $\Delta t^{(1)} = 6\Delta t^{(2)}$

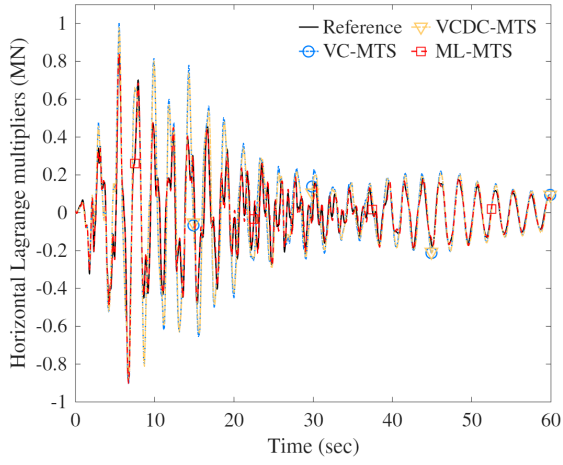


(e) Rotational DOF when $\Delta t^{(1)} = 10\Delta t^{(2)}$

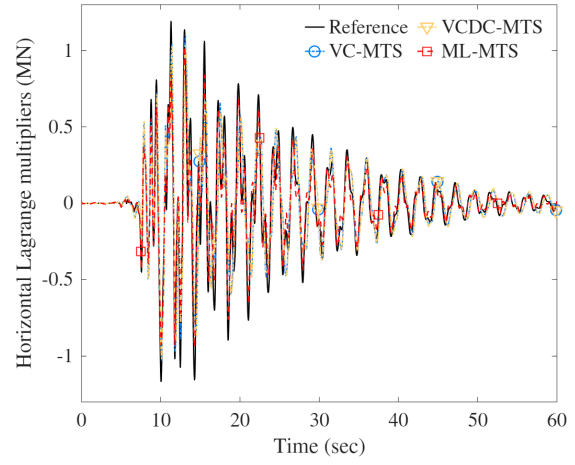


(f) Rotational DOF when $\Delta t^{(1)} = 6\Delta t^{(2)}$

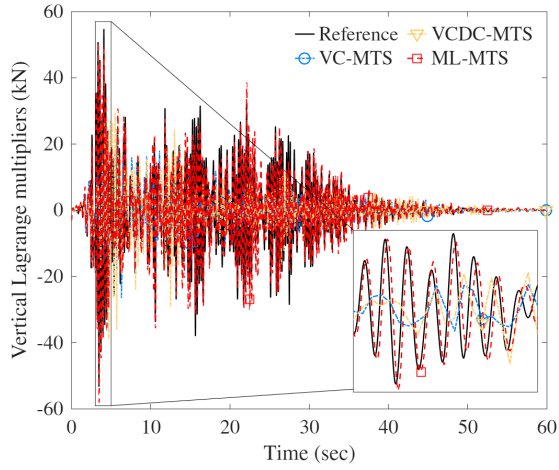
Fig. 7. Time history of Lagrange multipliers at the receiver in the nine-story building problem for the chirp signal.



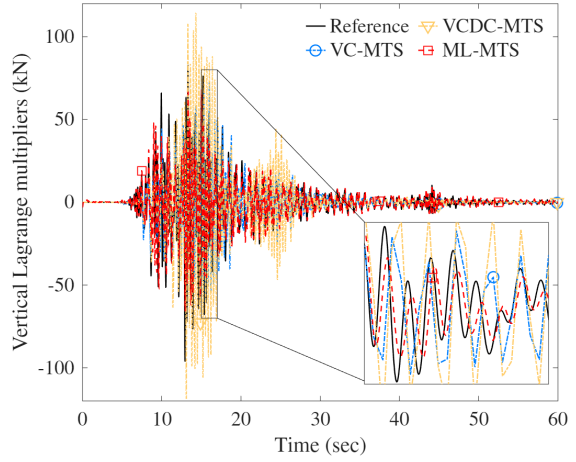
(a) Horizontal DOF for El Centro earthquake



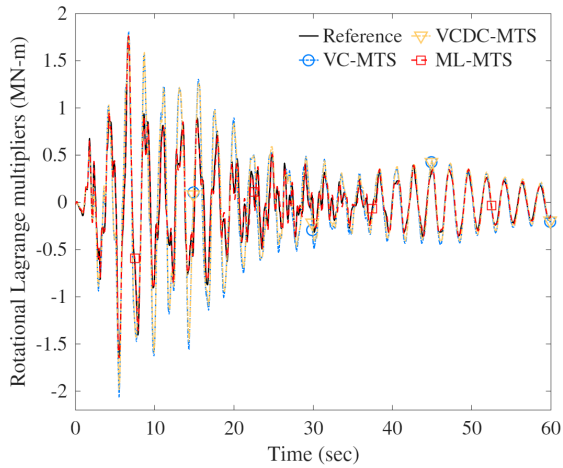
(b) Horizontal DOF for Kobe earthquake



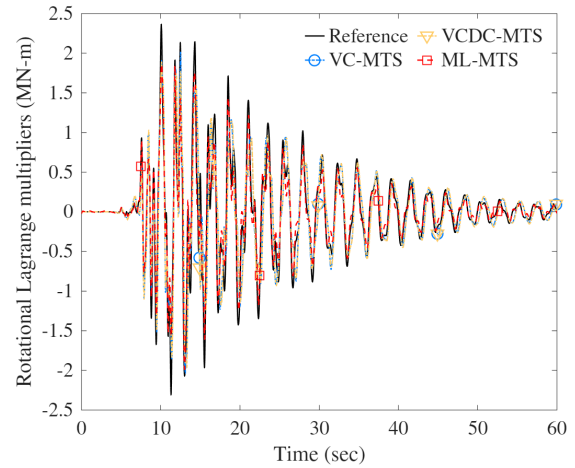
(c) Vertical DOF for El Centro earthquake



(d) Vertical DOF for Kobe earthquake

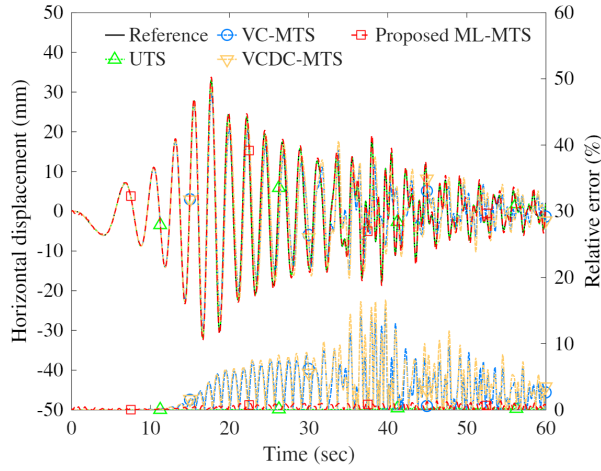


(e) Rotational DOF for El Centro earthquake

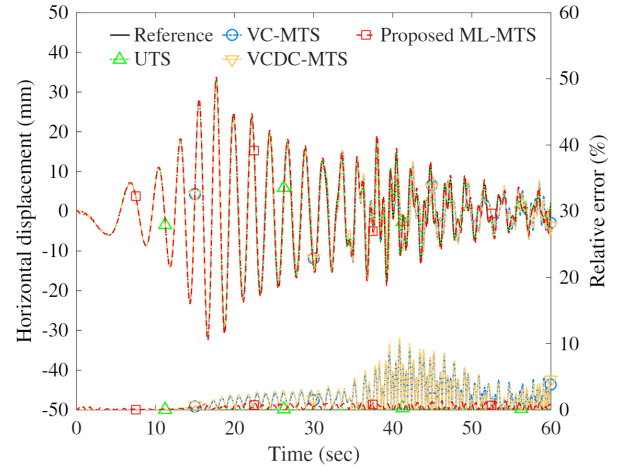


(f) Rotational DOF for Kobe earthquake

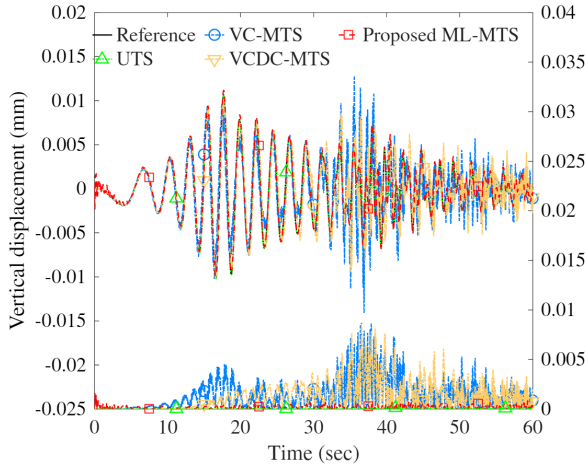
Fig. 8. Time history of Lagrange multipliers at the receiver in the nine-story building problem for two earthquake records with $\Delta t^{(1)} = 6\Delta t^{(2)}$.



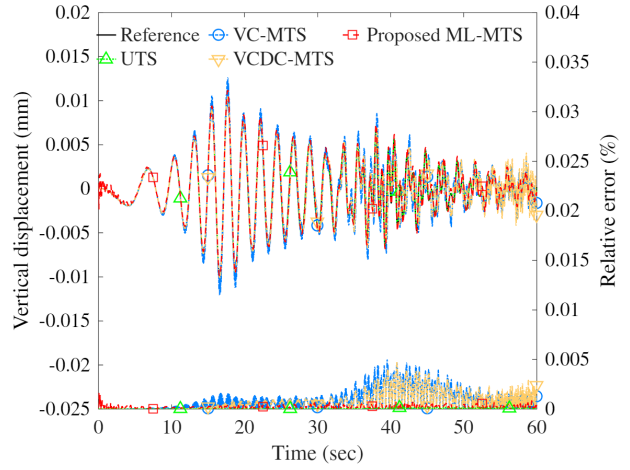
(a) Horizontal displacement when $\Delta t^{(1)} = 10\Delta t^{(2)}$



(b) Horizontal displacement when $\Delta t^{(1)} = 6\Delta t^{(2)}$

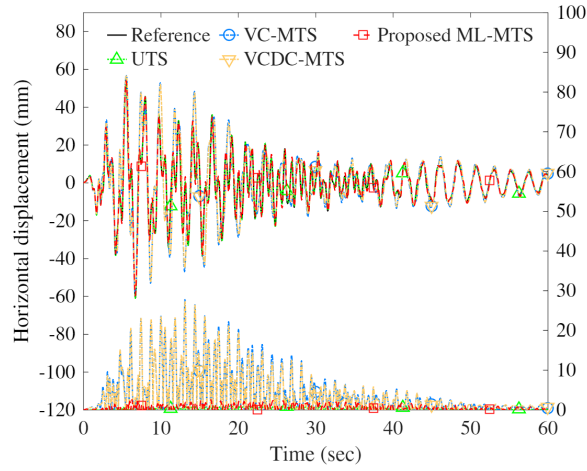


(c) Vertical displacement when $\Delta t^{(1)} = 10\Delta t^{(2)}$

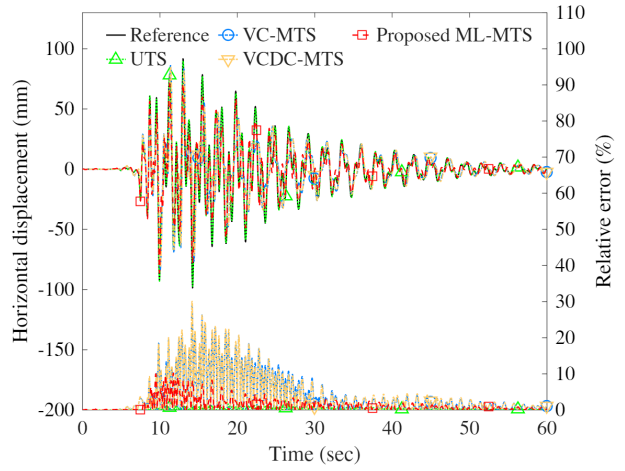


(d) Vertical displacement when $\Delta t^{(1)} = 6\Delta t^{(2)}$

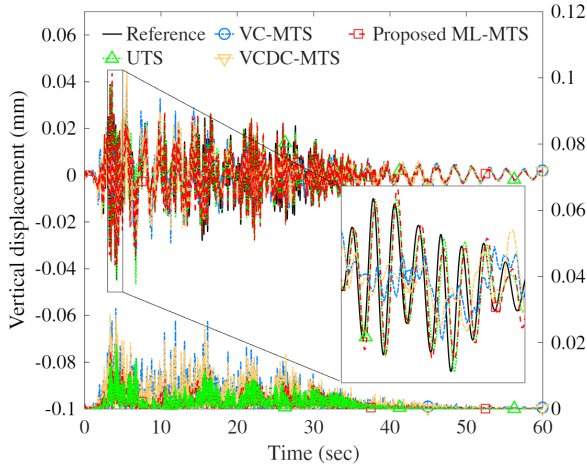
Fig. 9. Time history of horizontal and vertical displacements at the receiver in the nine-story building problem for the chirp signal. The upper curves represent displacements and the lower curves represent relative errors.



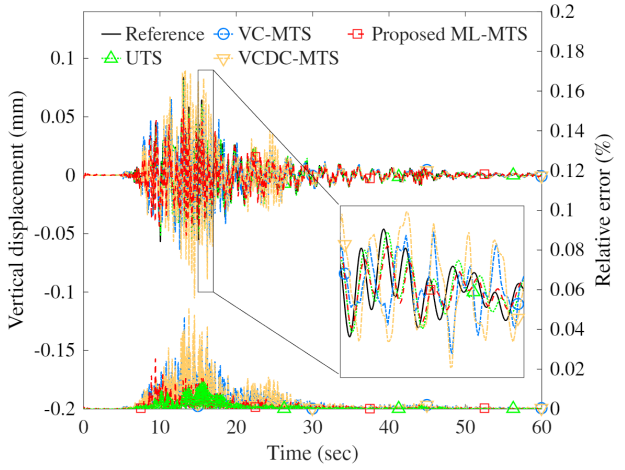
(a) Horizontal displacement for El Centro earthquake



(b) Horizontal displacement for Kobe earthquake



(c) Vertical displacement for El Centro earthquake



(d) Vertical displacement for Kobe earthquake

Fig. 10. Time history of horizontal and vertical displacements at the receiver in the nine-story building problem for two earthquake records with $\Delta t^{(1)} = 6\Delta t^{(2)}$. The upper curves represent displacements and the lower curves represent relative errors.

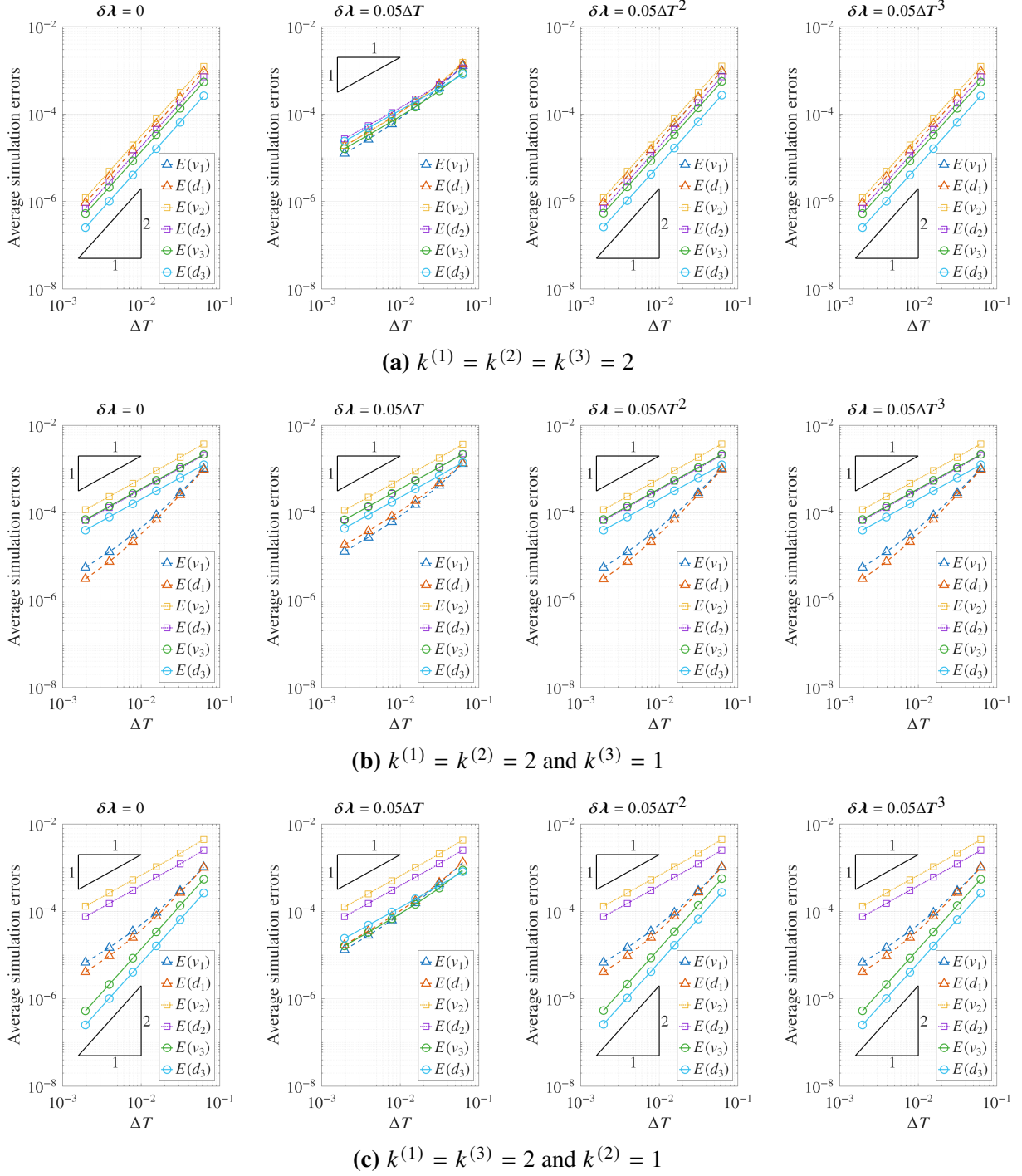


Fig. 11. Average simulation errors in velocities and displacements in the 3-DOF problem.